Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 07, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your local cluster. That is where the learning happens.

মূল্যায়ন নির্দেশিকা ও কুইক রেফারেন্স হ্যান্ডবুক

DEVOPS ENGINEERING MASTERY — Volume 07: Kubernetes & Container Orchestration

এই সহায়ক খণ্ডটি বইয়ের ২৪টি অধ্যায়ের শেখা যাচাই করার জন্য তৈরি। এতে রয়েছে ১২০টি বহুনির্বাচনি প্রশ্ন (MCQ), ৩০টি সংক্ষিপ্ত প্রশ্ন, ২০টি ট্রাবলশুটিং সিনারিও, ১০টি প্রোডাকশন কেস স্টাডি, একটি ব্যবহারিক ফাইনাল পরীক্ষা, সম্পূর্ণ উত্তর নির্দেশিকা, ১৬৫টি শব্দের গ্লসারি এবং kubectl, YAML, Helm ও Kustomize-এর কুইক রেফারেন্স। কাল্পনিক প্রসঙ্গ TechNova (technova.example, 203.0.113.x) — কোনো বাস্তব কোম্পানি, IP বা credential নয়।

কীভাবে ব্যবহার করবেন

- প্রতিটি অধ্যায় পড়ার পর সংশ্লিষ্ট MCQ গুচ্ছটি নিজে সমাধান করুন; উত্তর দেখার আগে অন্তত একবার চেষ্টা করুন।
- সংক্ষিপ্ত প্রশ্ন ও সিনারিও মুখস্থ না করে নিজের ভাষায় ব্যাখ্যা লিখুন — তারপর উত্তর নির্দেশিকার সঙ্গে মিলিয়ে দেখুন।
- ফাইনাল পরীক্ষাটি kind বা Minikube-এ একটি পরীক্ষামূলক ক্লাস্টারে, সময় ধরে দিন। প্রোডাকশন ক্লাস্টারে অনুশীলন করবেন না।
- গ্লসারি ও রেফারেন্স অংশটি কাজের সময় পাশে রাখার জন্য তৈরি — পরীক্ষার হলে নয়, ল্যাবে ব্যবহার করুন।

> **সতর্কতা:** Secret, kubeconfig বা token কখনোই উদাহরণের placeholder দিয়ে বাস্তব ক্লাস্টারে ব্যবহার করবেন না। `CHANGE-ME` লেখা মান সর্বদা বদলে নিন এবং আসল credential কখনো Git-এ commit করবেন না।

নম্বর বণ্টন — এক নজরে

অংশ	সংখ্যা	উদ্দেশ্য
MCQ	১২০	২৪ অধ্যায়ের ধারণা যাচাই
সংক্ষিপ্ত প্রশ্ন	৩০	ব্যাখ্যা ও যুক্তি যাচাই
ট্রাবলশুটিং সিনারিও	২০	সমস্যা নির্ণয় ও সমাধান

কেস স্টাডি	১০	প্রোডাকশন ঘটনা থেকে শিক্ষা
ফাইনাল পরীক্ষা	৩ ঘণ্টা	তত্ত্ব + হাতে-কলমে ক্লাস্টার কাজ
গ্লসারি	১৬৫	পরিভাষা ও ব্যবহারিক প্রসঙ্গ

কঠিনতার স্তর: **[Beginner]** — মৌলিক ধারণা; **[Intermediate]** — প্রয়োগ ও তুলনা; **[Advanced]** — নকশা, ট্রেড-অফ ও প্রোডাকশন বিচার।

বহুনির্বাচনি প্রশ্ন (MCQ) — ১২০টি

প্রতিটি প্রশ্নের চারটি বিকল্প (ক, খ, গ, ঘ)। উত্তর এই অংশে দেওয়া হয়নি — উত্তর নির্দেশিকা অংশে দেখুন।

অধ্যায় ১–৩: আর্কিটেকচার, API Server / etcd / Scheduler

1. Kubernetes-এ control plane-এর কোন কম্পোনেন্টটি ক্লাস্টারের সব state-এর একমাত্র সত্যের উৎস (source of truth) হিসেবে কাজ করে? [Beginner]

1. ক) kube-scheduler
2. খ) etcd
3. গ) kube-proxy
4. ঘ) kubelet

2. API server-এর প্রধান কাজ কোনটি? [Beginner]

1. ক) Pod-এ কনটেইনার চালানো
2. খ) সব অনুরোধ যাচাই, প্রমাণীকরণ ও অনুমোদন করে etcd-তে state লেখা
3. গ) নোডের মধ্যে নেটওয়ার্ক রাউটিং করা
4. ঘ) কনটেইনার ইমেজ বিল্ড করা

3. kube-scheduler Pod-কে নোডে বসানোর সময় প্রথমে কী করে? [Intermediate]

1. ক) সব নোডে Pod চালিয়ে দেখে
2. খ) filtering (যেসব নোড শর্ত পূরণ করে না বাদ দেওয়া), তারপর scoring করে সেরা নোড বেছে নেওয়া
3. গ) etcd সরাসরি লেখা
4. ঘ) kube-proxy-কে জানানো

4. etcd-এ quorum হারালে কী ঘটে? [Advanced]

1. ক) চলমান Pod-গুলো সঙ্গে সঙ্গে বন্ধ হয়ে যায়
2. খ) ক্লাস্টার read-only হয়ে যায় — নতুন লেখা বন্ধ, তবে চলমান workload চলতে থাকে
3. গ) সব Service IP বদলে যায়
4. ঘ) kubelet নিজে Pod তৈরি করতে শুরু করে

5. কোনটি worker node-এর কম্পোনেন্ট নয়? [Beginner]

1. ক) kubelet
2. খ) kube-proxy
3. গ) container runtime
4. ঘ) kube-controller-manager

অধ্যায় ৪–৫: Pod

6. Pod হলো Kubernetes-এর — [Beginner]

1. ক) সবচেয়ে ছোট deployable একক, এক বা একাধিক কনটেইনারের সমষ্টি
2. খ) একটি নেটওয়ার্ক প্লাগইন
3. গ) একটি storage class
4. ঘ) একটি DNS সার্ভার

7. একই Pod-এর কনটেইনারগুলো কী ভাগ করে নেয়? [Beginner]

1. ক) শুধু CPU
2. খ) network namespace (একই IP) ও চাইলে volume
3. গ) আলাদা আলাদা IP
4. ঘ) আলাদা নোড

8. Init container-এর কাজ কী? [Intermediate]

1. ক) মূল কনটেইনার বন্ধ হলে পুনরায় চালু করা
2. খ) মূল কনটেইনার শুরুর আগে সেটআপ কাজ (যেমন ডেটাবেসের জন্য অপেক্ষা) সম্পন্ন করা
3. গ) লগ সংগ্রহ করা
4. ঘ) Pod-কে অন্য নোডে সরানো

9. Pod-এর `restartPolicy: Never` কখন উপযুক্ত? [Intermediate]

1. ক) দীর্ঘমেয়াদি ওয়েব সার্ভারে
2. খ) একবার চালিয়ে শেষ হওয়া Job-এর Pod-এ
3. গ) StatefulSet-এ
4. ঘ) DaemonSet-এ

10. Pod `Pending` অবস্থায় আটকে থাকার সবচেয়ে সাধারণ কারণ কোনটি? [Intermediate]

1. ক) ইমেজ ট্যাগ ভুল
2. খ) scheduler উপযুক্ত নোড খুঁজে পায়নি (resource不足, nodeSelector, taint ইত্যাদি)
3. গ) liveness probe ব্যর্থ
4. ঘ) Service-এর selector ভুল

অধ্যায় ৬–৭: Deployment, StatefulSet, DaemonSet, Job

11. Deployment কী পরিচালনা করে? [Beginner]

1. ক) সরাসরি Pod
2. খ) ReplicaSet-এর মাধ্যমে Pod-এর replica ও rolling update
3. গ) PersistentVolume

4. ঘ) Ingress rule

12. Rolling update-এ `maxUnavailable: 0` ও `maxSurge: 1` মানে কী? [Intermediate]

1. ক) কোনো Pod বন্ধ না করে একবারে একটি অতিরিক্ত Pod তৈরি করে আপডেট
2. খ) সব Pod একসঙ্গে বন্ধ করে আপডেট
3. গ) আপডেট বন্ধ থাকে
4. ঘ) শুধু একটি Pod আপডেট হয়

13. StatefulSet কখন Deployment-এর চেয়ে উপযুক্ত? [Intermediate]

1. ক) stateless ওয়েব অ্যাপে
2. খ) স্থিতিশীল নেটওয়ার্ক পরিচয়, ক্রমিক চালু/বন্ধ ও প্রতিটি Pod-এর নিজস্ব storage দরকার হলে
3. গ) প্রতি নোডে একটি Pod চালাতে হলে
4. ঘ) একবার চলা ব্যাচ কাজে

14. DaemonSet-এর মূল উদ্দেশ্য কী? [Beginner]

1. ক) নির্দিষ্ট সংখ্যক replica চালানো
2. খ) (উপযুক্ত) প্রতিটি নোডে একটি করে Pod চালানো, যেমন লগ বা মনিটরিং এজেন্ট
3. গ) সময়সূচি অনুযায়ী কাজ চালানো
4. ঘ) ডেটাবেস ব্যাকআপ নেওয়া

15. CronJob ও Job-এর পার্থক্য কী? [Beginner]

1. ক) কোনো পার্থক্য নেই
2. খ) Job একবার সম্পন্ন হওয়ার কাজ; CronJob সময়সূচি অনুযায়ী বারবার Job তৈরি করে
3. গ) CronJob শুধু StatefulSet চালায়
4. ঘ) Job শুধু রাতে চলে

অধ্যায় ৮–৯: Namespace, Label, Selector

16. Namespace-এর প্রধান উদ্দেশ্য কী? [Beginner]

1. ক) নেটওয়ার্ক আলাদা করা
2. খ) একই ক্লাস্টারে রিসোর্সের নাম ও নীতি অনুযায়ী যৌক্তিক বিভাজন (যেমন dev/staging/prod)
3. গ) Pod-এর IP বদলানো
4. ঘ) ইমেজ সংরক্ষণ

17. Label ও annotation-এর পার্থক্য কী? [Intermediate]

1. ক) দুটোই selector-এ ব্যবহার করা যায়
2. খ) Label পরিচয় ও নির্বাচনের জন্য (selector-এ ব্যবহৃত হয়); annotation তথ্য/মেটাডেটার জন্য, selector-এ ব্যবহৃত হয় না
3. গ) annotation শুধু Pod-এ যায়

4. ঘ) Label শুধু Secret-এ যায়

18. Service কীভাবে সঠিক Pod খুঁজে পায়? [Beginner]

1. ক) Pod-এর নাম দিয়ে
2. খ) label selector মিলিয়ে Endpoints তৈরি করে
3. গ) IP রেঞ্জ দিয়ে
4. ঘ) namespace-এর নাম দিয়ে

19. `kubectl get pods -l app=technova,tier=web` কমান্ডটি কী করে? [Beginner]

1. ক) সব Pod মুছে ফেলে
2. খ) দুটি label-ই মেলে এমন Pod দেখায়
3. গ) নতুন Pod তৈরি করে
4. ঘ) Pod-এর লগ দেখায়

20. ResourceQuota কোন স্তরে প্রয়োগ হয়? [Intermediate]

1. ক) ক্লাস্টার স্তরে
2. খ) namespace স্তরে — ওই namespace-এর মোট resource ব্যবহার সীমিত করে
3. গ) Pod স্তরে
4. ঘ) নোড স্তরে

অধ্যায় ১০–১১: ConfigMap ও Secret

21. ConfigMap কখন ব্যবহার করবেন? [Beginner]

1. ক) পাসওয়ার্ড সংরক্ষণে
2. খ) সংবেদনশীল নয় এমন কনফিগারেশন (যেমন APP_MODE=production) Pod-এ দেওয়ার জন্য
3. গ) ইমেজ সংরক্ষণে
4. ঘ) নেটওয়ার্ক নীতি লেখায়

22. Secret-এর ডেটা etcd-তে কীভাবে সংরক্ষিত হয়, ডিফল্টভাবে? [Intermediate]

1. ক) শক্তিশালী encryption-সহ
2. খ) base64-encoded, কিন্তু ডিফল্টে encrypt করা নয় (encryption at rest আলাদাভাবে চালু করতে হয়)
3. গ) plain text-এ, সবাই পড়তে পারে
4. ঘ) কখনোই etcd-তে যায় না

23. ConfigMap-কে Pod-এ দেওয়ার দুটি প্রধান উপায় কী? [Beginner]

1. ক) environment variable ও volume mount
2. খ) Service ও Ingress
3. গ) label ও annotation

4. ঘ) CronJob ও Job

24. চলমান Pod-এ ConfigMap বদলালে কী ঘটে, volume mount করা থাকলে? [Advanced]

1. ক) Pod সঙ্গে সঙ্গে restart হয়
2. খ) kubelet কিছুক্ষণ পর ফাইল আপডেট করে, কিন্তু অ্যাপকে নিজে reload করতে হয়; env var-এর ক্ষেত্রে Pod restart ছাড়া বদলায় না
3. গ) কিছুই বদলায় না, কখনো নয়
4. ঘ) Deployment স্বয়ংক্রিয়ভাবে rollout করে

25. Secret ব্যবহারের সেরা অনুশীলন কোনটি? [Intermediate]

1. ক) Secret YAML Git-এ commit করা
2. খ) RBAC দিয়ে পড়ার অধিকার সীমিত করা ও বাহ্যিক secret manager ব্যবহার করা
3. গ) Secret-কে ConfigMap-এ রূপান্তর করা
4. ঘ) সব namespace-এ একই Secret কপি করা

অধ্যায় ১২–১৩: Resource, Request/Limit, HPA

26. Pod-এ CPU `requests` ও `limits`-এর ভূমিকা কী? [Beginner]

1. ক) দুটোই একই — Pod কতটুকু পাবে তা ঠিক করে
2. খ) requests = scheduler-এর জন্য ন্যূনতম গ্যারান্টি; limits = সর্বোচ্চ সীমা, ছাড়ালে throttle/OOM হতে পারে
3. গ) requests শুধু মেমরির জন্য
4. ঘ) limits শুধু লগের জন্য

27. মেমরি limit ছাড়িয়ে গেলে Pod-এর কী হয়? [Intermediate]

1. ক) কিছুই হয় না
2. খ) কনটেইনার OOMKilled হয়ে restart হতে পারে
3. গ) Pod অন্য নোডে চলে যায়
4. ঘ) CPU বাড়িয়ে দেওয়া হয়

28. HPA (Horizontal Pod Autoscaler) কীসের ভিত্তিতে replica বাড়ায়/কমায়? [Beginner]

1. ক) নোডের সংখ্যা
2. খ) CPU/memory ব্যবহার বা custom metric-এর ভিত্তিতে
3. গ) দিনের সময়
4. ঘ) ইমেজের আকার

29. HPA কাজ করার জন্য কোনটি আবশ্যক? [Intermediate]

1. ক) metrics-server (বা সমতুল্য metrics API) চালু থাকা
2. খ) Ingress থাকা
3. গ) StatefulSet থাকা

4. ঘ) Pod-এ কোনো resource request না থাকা

30. QoS class `Guaranteed` কখন পাওয়া যায়? [Advanced]

1. ক) কোনো request/limit না দিলে
2. খ) প্রতিটি কনটেইনারে CPU ও memory-র request = limit হলে
3. গ) শুধু limit দিলে
4. ঘ) শুধু request দিলে

অধ্যায় ১৪–১৫: Networking ও CNI

31. Kubernetes networking মডেলের মূল নিয়ম কোনটি? [Beginner]

1. ক) প্রতিটি Pod-এর নিজস্ব IP থাকে এবং সব Pod NAT ছাড়াই একে অপরের সঙ্গে কথা বলতে পারে
2. খ) শুধু একই নোডের Pod কথা বলতে পারে
3. গ) Pod-এর কোনো IP থাকে না
4. ঘ) সব ট্রাফিক Ingress দিয়ে যেতে হয়

32. CNI (Container Network Interface)-এর কাজ কী? [Intermediate]

1. ক) কনটেইনার ইমেজ টানা
2. খ) Pod-এর নেটওয়ার্ক ইন্টারফেস তৈরি ও IP বরাদ্দের প্লাগইন কাঠামো দেওয়া
3. গ) DNS রেকর্ড লেখা
4. ঘ) Secret encrypt করা

33. kube-proxy-এর মূল কাজ কী? [Beginner]

1. ক) Pod চালানো
2. খ) Service-এর ClusterIP-তে আসা ট্রাফিক সঠিক Pod-এ পাঠানোর নিয়ম (iptables/IPVS) রক্ষণাবেক্ষণ
3. গ) etcd ব্যাকআপ নেওয়া
4. ঘ) Ingress TLS বন্ধ করা

34. Pod-এর IP বদলে গেলে Service কেন ভাঙে না? [Intermediate]

1. ক) Pod-এর IP কখনো বদলায় না
2. খ) Service স্থিতিশীল ClusterIP ও Endpoints-এর মাধ্যমে Pod-কে খুঁজে পায়
3. গ) DNS IP মনে রাখে
4. ঘ) kubelet IP স্থির রাখে

35. NetworkPolicy ডিফল্টভাবে কী করে? [Advanced]

1. ক) কিছু না — NetworkPolicy না থাকলে সব ট্রাফিক অনুমোদিত; একটি Policy নির্বাচিত Pod-কে সীমিত করে
2. খ) সব ট্রাফিক বন্ধ করে দেয়
3. গ) শুধু বাইরের ট্রাফিক বন্ধ করে

4. ঘ) DNS বন্ধ করে দেয়

অধ্যায় ১৬–১৭: Service ও DNS

36. ClusterIP Service কাদের জন্য? [Beginner]

1. ক) ইন্টারনেট থেকে সরাসরি আসা ট্রাফিকের জন্য
2. খ) শুধু ক্লাস্টারের ভেতরের Pod-গুলোর মধ্যে যোগাযোগের জন্য
3. গ) বাইরের ডেটাবেসের জন্য
4. ঘ) নোডের SSH-এর জন্য

37. NodePort Service কী করে? [Beginner]

1. ক) প্রতিটি নোডে একটি পোর্ট (30000–32767) খুলে বাইরে থেকে ট্রাফিক Pod-এ পাঠায়
2. খ) শুধু DNS দেয়
3. গ) Pod-এর সংখ্যা বাড়ায়
4. ঘ) TLS বন্ধ করে

38. Headless Service (`clusterIP: None`) কখন দরকার? [Intermediate]

1. ক) LoadBalancer চাইলে
2. খ) প্রতিটি Pod-এর IP সরাসরি DNS-এ জানাতে হলে, যেমন StatefulSet-এ
3. গ) Ingress চাইলে
4. ঘ) Pod বন্ধ করতে হলে

39. CoreDNS-এ `technova-api.default.svc.cluster.local` কী বোঝায়? [Intermediate]

1. ক) একটি বাইরের ওয়েবসাইট
2. খ) default namespace-এর technova-api Service-এর সম্পূর্ণ DNS নাম
3. গ) একটি Pod-এর নাম
4. ঘ) একটি নোডের hostname

40. একই namespace-এর Pod অন্য Service-কে কীভাবে ডাকে? [Beginner]

1. ক) শুধু Service-এর নামে (যেমন `http://technova-api:8080`)
2. খ) নোডের IP দিয়ে
3. গ) Pod-এর IP দিয়ে
4. ঘ) Ingress দিয়ে

অধ্যায় ১৮–১৯: Ingress ও Gateway API

41. Ingress-এর মূল কাজ কী? [Beginner]

1. ক) Pod তৈরি করা
2. খ) HTTP/HTTPS ট্রাফিক host ও path অনুযায়ী বিভিন্ন Service-এ রাউট করা

3. গ) ডেটাবেস ব্যাকআপ
4. ঘ) Secret তৈরি করা

42. Ingress resource একা কাজ করে কি? [Intermediate]

1. ক) হ্যাঁ, সবসময়
2. খ) না — একটি Ingress controller (যেমন NGINX) চালু থাকতে হয়, যা rule পড়ে ট্রাফিক রাউট করে
3. গ) শুধু ক্লাউডে
4. ঘ) শুধু StatefulSet-এ

43. Ingress-এ TLS বন্ধ করতে কী দরকার? [Intermediate]

1. ক) একটি TLS Secret ও `tls` অংশে host উল্লেখ
2. খ) শুধু Service-এর নাম
3. গ) একটি ConfigMap
4. ঘ) একটি Job

44. Gateway API Ingress-এর তুলনায় কী সুবিধা দেয়? [Advanced]

1. ক) এটি Pod চালায়
2. খ) ভূমিকা-ভিত্তিক (GatewayClass/Gateway/HTTPRoute) আলাদা দায়িত্ব, ভালো এক্সটেনশন ও ক্রস-namespace রাউটিং
3. গ) এটি DNS বদলে দেয়
4. ঘ) এটি etcd বদলে দেয়

45. Ingress-এ 404 পেলে প্রথমে কী যাচাই করবেন? [Intermediate]

1. ক) Pod-এর CPU
2. খ) host/path rule ও Service-এর নাম-পোর্ট মিলছে কি না
3. গ) etcd-এর আকার
4. ঘ) নোডের disk

অধ্যায় ২০: PV, PVC, StorageClass, CSI

46. PersistentVolume (PV) ও PersistentVolumeClaim (PVC)-এর সম্পর্ক কী? [Beginner]

1. ক) PV হলো storage-এর চাহিদা, PVC হলো সরবরাহ
2. খ) PV হলো ক্লাস্টারে storage-এর সরবরাহ; PVC হলো ব্যবহারকারীর চাহিদা, যা PV-এর সঙ্গে bind হয়
3. গ) দুটো একই জিনিস
4. ঘ) PVC শুধু ConfigMap-এর জন্য

47. StorageClass কী করে? [Intermediate]

1. ক) Pod-এর সংখ্যা নির্ধারণ করে
2. খ) PVC-এর চাহিদা অনুযায়ী স্বয়ংক্রিয়ভাবে (dynamic provisioning) PV তৈরি করার নিয়ম দেয়

3. গ) DNS রেকর্ড তৈরি করে
4. ঘ) Secret encrypt করে

48. CSI driver-এর ভূমিকা কী? [Intermediate]

1. ক) কনটেইনার ইমেজ বিল্ড করা
2. খ) বাইরের storage সিস্টেমকে Kubernetes-এর সঙ্গে যুক্ত করার মানসম্মত ইন্টারফেস দেওয়া
3. গ) নেটওয়ার্ক পলিসি লেখা
4. ঘ) Pod scheduling করা

49. PVC `Pending` থাকার একটি সাধারণ কারণ কী? [Intermediate]

1. ক) Pod-এর imagePullBackOff
2. খ) উপযুক্ত StorageClass/PV নেই বা provisioner ব্যর্থ হয়েছে
3. গ) Ingress rule ভুল
4. ঘ) liveness probe ব্যর্থ

50. `accessModes: ReadWriteOnce` মানে কী? [Beginner]

1. ক) অনেক নোড থেকে একসঙ্গে লেখা যায়
2. খ) একটিমাত্র নোড থেকে read-write mount করা যায়
3. গ) শুধু পড়া যায়
4. ঘ) কেউ mount করতে পারে না

অধ্যায় ২১: RBAC

51. RBAC-এ Role, ClusterRole, RoleBinding ও ClusterRoleBinding-এর মধ্যে পার্থক্য কী? [Intermediate]

1. ক) সব একই
2. খ) Role/RoleBinding namespace-এর মধ্যে; ClusterRole/ClusterRoleBinding ক্লাস্টারজুড়ে বা একাধিক namespace-এ
3. গ) Role শুধু Pod-এর জন্য
4. ঘ) Binding শুধু ServiceAccount-এর জন্য নয়

52. ServiceAccount কী? [Beginner]

1. ক) একজন মানুষের লগইন
2. খ) Pod বা অ্যাপ্লিকেশনের পরিচয়, যাকে RBAC দিয়ে অনুমতি দেওয়া হয়
3. গ) একটি Secret
4. ঘ) একটি namespace

53. `kubectl auth can-i create deployments --namespace=prod` কী যাচাই করে? [Intermediate]

1. ক) Deployment তৈরি হয়েছে কি না
2. খ) বর্তমান ব্যবহারকারী prod namespace-এ Deployment তৈরি করতে পারবে কি না

3. গ) Pod চলছে কি না
4. ঘ) ক্লাস্টারের সংস্করণ

54. সর্বনিম্ন সুযোগের নীতি (least privilege) মানে কী? [Intermediate]

1. ক) সবাইকে cluster-admin দেওয়া
2. খ) কাজের জন্য যতটুকু দরকার, ঠিক ততটুকু অনুমতিই দেওয়া
3. গ) কোনো অনুমতি না দেওয়া
4. ঘ) শুধু read অনুমতি দেওয়া

55. Pod-এ `automountServiceAccountToken: false` দেওয়া কেন নিরাপত্তার জন্য ভালো? [Advanced]

1. ক) এতে Pod দ্রুত চলে
2. খ) API-তে কথা বলার দরকার না থাকা Pod-এ token না থাকলে token চুরি/অপব্যবহারের ঝুঁকি কমে
3. গ) এতে Pod-এর IP বদলায়
4. ঘ) এতে Secret মুছে যায়

অধ্যায় ২২: Pod Security

56. Pod Security Standards-এর `restricted` প্রোফাইলে কী আশা করা হয়? [Intermediate]

1. ক) root হিসেবে চলা
2. খ) non-root ব্যবহারকারী, read-only root filesystem, capability বাদ দেওয়া ইত্যাদি কঠোর নিয়ম
3. গ) privileged কনটেইনার
4. ঘ) host network ব্যবহার

57. `runAsNonRoot: true` কী নিশ্চিত করে? [Beginner]

1. ক) Pod root হিসেবে চলবে না
2. খ) Pod দ্রুত চলবে
3. গ) Pod-এর নেটওয়ার্ক বন্ধ হবে
4. ঘ) Pod restart হবে না

58. `allowPrivilegeEscalation: false` কী বন্ধ করে? [Intermediate]

1. ক) Pod-এর লগ
2. খ) প্রক্রিয়ার root-এর চেয়ে বেশি সুযোগ পাওয়ার পথ
3. গ) Pod-এর restart
4. ঘ) Service-এর সংযোগ

59. ImagePullPolicy `Always` কখন গুরুত্বপূর্ণ? [Intermediate]

1. ক) `:latest` ট্যাগ ব্যবহার করলে, যাতে পুরনো cached ইমেজ না চলে
2. খ) সবসময়, সব ট্যাগে

3. গ) শুধু Job-এ
4. ঘ) কখনো নয়

60. Admission controller (যেমন Pod Security Admission) কোথায় কাজ করে? [Advanced]

1. ক) kubelet-এ
2. খ) API server-এ — অনুরোধ etcd-তে যাওয়ার আগে যাচাই/পরিবর্তন করে
3. গ) etcd-তে
4. ঘ) CoreDNS-এ

অধ্যায় ২৩–২৪ (প্রথমার্ধ): Troubleshooting ও Monitoring

61. Pod `CrashLoopBackOff` দেখলে প্রথমে কোন কমান্ডটি সবচেয়ে তথ্যপূর্ণ? [Beginner]

1. ক) `kubectl get nodes`
2. খ) `kubectl logs <pod>` ও `kubectl describe pod <pod>`
3. গ) `kubectl get ingress`
4. ঘ) `kubectl delete pod`

62. `ImagePullBackOff` মানে কী? [Beginner]

1. ক) Pod crash করছে
2. খ) কনটেইনার ইমেজ টানা যাচ্ছে না — নাম/ট্যাগ ভুল, registry-তে নেই, বা credential নেই
3. গ) Pod-এর CPU শেষ
4. ঘ) Service-এর selector ভুল

63. Liveness probe ও readiness probe-এর পার্থক্য কী? [Intermediate]

1. ক) দুটো একই
2. খ) Liveness ব্যর্থ হলে kubelet কনটেইনার restart করে; readiness ব্যর্থ হলে Pod Service-এর Endpoint থেকে সরিয়ে নেওয়া হয়
3. গ) Readiness ব্যর্থ হলে Pod মুছে যায়
4. ঘ) Liveness শুধু Job-এ ব্যবহার হয়

64. `kubectl describe pod`-এর Events অংশে কী খুঁজবেন? [Intermediate]

1. ক) শুধু Pod-এর নাম
2. খ) FailedScheduling, FailedMount, BackOff, Unhealthy ইত্যাদি সতর্কবার্তা ও কারণ
3. গ) Pod-এর ইমেজের আকার
4. ঘ) ক্লাস্টারের সংস্করণ

65. Prometheus-এ Kubernetes-এর metric সংগ্রহের সাধারণ পদ্ধতি কী? [Intermediate]

1. ক) Pod-এর লগ পড়া
2. খ) `/metrics` endpoint থেকে pull করা (scrape), প্রায়ই ServiceMonitor দিয়ে

3. গ) etcd সরাসরি পড়া
4. ঘ) Ingress দিয়ে পাঠানো

66. কোনটি observability-র তিন স্তম্ভের একটি? [Beginner]

1. ক) Deployment
2. খ) Logs, metrics ও traces
3. গ) ConfigMap
4. ঘ) Namespace

67. Pod-এর লগ হঠাৎ বন্ধ হয়ে গেলে প্রথমে কী যাচাই করবেন? [Intermediate]

1. ক) Pod এখনো চলছে কি না ও restart হয়েছে কি না (`--previous` লগ দেখুন)
2. খ) Ingress rule
3. গ) StorageClass
4. ঘ) RBAC Role

68. `kubectl top pod` কাজ না করলে সম্ভাব্য কারণ কী? [Intermediate]

1. ক) Ingress নেই
2. খ) metrics-server ইনস্টল/চালু নেই
3. গ) Pod-এ label নেই
4. ঘ) Service নেই

69. একটি Deployment rollout আটকে গেলে কোন কমান্ডে ইতিহাস দেখা যায়? [Beginner]

1. ক) `kubectl rollout history deployment/technova-app`
2. খ) `kubectl get nodes`
3. গ) `kubectl logs deployment/technova-app`
4. ঘ) `kubectl describe service technova-app`

70. Alerting-এ সবচেয়ে কার্যকর পদ্ধতি কোনটি? [Advanced]

1. ক) সব metric-এ alert দেওয়া
2. খ) ব্যবহারকারীর ওপর প্রভাব ফেলে এমন লক্ষণ (যেমন error rate, latency) ও কম শব্দে, কার্যকর alert
3. গ) শুধু CPU-তে alert দেওয়া
4. ঘ) alert না দেওয়া

অধ্যায় ২৪ (দ্বিতীয়ার্ধ): Helm, Kustomize, Upgrade/Backup/DR

71. Helm chart কী? [Beginner]

1. ক) একটি কনটেইনার ইমেজ
2. খ) Kubernetes manifest-এর টেমপ্লেটসহ প্যাকেজ (Chart.yaml, values.yaml, templates/)

3. গ) একটি DNS সার্ভার
4. ঘ) একটি storage driver

72. `helm upgrade --install` কী করে? [Intermediate]

1. ক) শুধু নতুন release ইনস্টল করে
2. খ) release না থাকলে ইনস্টল করে, থাকলে আপগ্রেড করে — idempotent পদ্ধতি
3. গ) release মুছে ফেলে
4. ঘ) chart ডাউনলোড করে

73. Helm release-এর revision কী কাজে লাগে? [Intermediate]

1. ক) ইমেজ বিল্ডে
2. খ) আগের সংস্করণে rollback (`helm rollback`) করতে
3. গ) Secret encrypt করতে
4. ঘ) Pod scheduling-এ

74. Kustomize-এর মূল ধারণা কী? [Beginner]

1. ক) টেমপ্লেট ভাষা দিয়ে manifest তৈরি
2. খ) টেমপ্লেট ছাড়াই base manifest-এর ওপর overlay/patch দিয়ে পরিবেশভেদে পরিবর্তন
3. গ) ইমেজ বিল্ড করা
4. ঘ) ক্লাস্টার আপগ্রেড করা

75. Kustomize-এ `kustomization.yaml`-এ কী থাকে? [Intermediate]

1. ক) শুধু ইমেজের নাম
2. খ) resources, image, commonLabels, patches ইত্যাদির তালিকা
3. গ) শুধু Secret
4. ঘ) শুধু Ingress rule

76. ক্লাস্টার আপগ্রেডের নিরাপদ ক্রম কোনটি? [Advanced]

1. ক) worker নোড আগে, control plane পরে
2. খ) control plane আগে (একটি minor version), তারপর worker নোড, API compatibility যাচাই করে
3. গ) সব একসঙ্গে
4. ঘ) etcd আগে মুছে ফেলা

77. etcd ব্যাকআপের আগে কী করা উচিত? [Intermediate]

1. ক) সব Pod মুছে ফেলা
2. খ) `etcdctl snapshot save` দিয়ে snapshot নেওয়া ও তা নিরাপদ জায়গায় রাখা, পুনরুদ্ধার পরীক্ষা করা
3. গ) Ingress মুছে ফেলা
4. ঘ) কিছুই না

78. Disaster recovery পরিকল্পনায় RTO ও RPO কী বোঝায়? [Intermediate]

1. ক) RTO = কত দ্রুত ফিরতে হবে; RPO = কতটুকু ডেটা হারানো সহ্য করা যায়
2. খ) RTO = Pod-এর সংখ্যা; RPO = নোডের সংখ্যা
3. গ) দুটোই একই
4. ঘ) RTO = ব্যাকআপের আকার

79. Velero-জাতীয় টুল সাধারণত কী ব্যাকআপ নেয়? [Intermediate]

1. ক) শুধু ইমেজ
2. খ) ক্লাস্টারের resource ও PersistentVolume-এর ডেটা
3. গ) শুধু লগ
4. ঘ) শুধু DNS

80. আপগ্রেডের আগে সবচেয়ে গুরুত্বপূর্ণ যাচাই কোনটি? [Advanced]

1. ক) ইমেজের ট্যাগ
2. খ) ব্যবহৃত API version-গুলো নতুন সংস্করণে সরানো (deprecated/removed) হয়েছে কি না
3. গ) Pod-এর label
4. ঘ) Ingress-এর host

অধ্যায় ২৪ (তৃতীয়াংশ): Managed Kubernetes ও GitOps

81. Managed Kubernetes (EKS/AKS/GKE)-এ ক্লাউড প্রদানকারী সাধারণত কী পরিচালনা করে? [Beginner]

1. ক) আপনার অ্যাপের কোড
2. খ) control plane (API server, etcd ইত্যাদি) — আপনি worker নোড ও workload পরিচালনা করেন
3. গ) আপনার Secret-এর মান
4. ঘ) আপনার Git repository

82. GitOps-এর মূল নীতি কী? [Intermediate]

1. ক) kubectl দিয়ে সরাসরি ক্লাস্টারে বদলানো
2. খ) Git-ই ক্লাস্টারের desired state-এর একমাত্র উৎস; একটি agent Git-এর সঙ্গে ক্লাস্টারকে মিলিয়ে রাখে
3. গ) Git-এ Secret রাখা
4. ঘ) হাতে হাতে YAML লেখা

83. Argo CD-তে `OutOfSync` মানে কী? [Intermediate]

1. ক) অ্যাপ মুছে গেছে
2. খ) ক্লাস্টারের অবস্থা Git-এর ঘোষিত অবস্থার সঙ্গে মিলছে না
3. গ) Git repository নেই
4. ঘ) Pod crash করছে

84. GitOps-এ Secret কীভাবে নিরাপদে রাখা হয়? [Advanced]

1. ক) plain text-এ Git-এ
2. খ) Sealed Secrets বা External Secrets-এর মতো পদ্ধতিতে encrypt/বাহ্যিক উৎস থেকে আনা হয়
3. গ) ConfigMap-এ
4. ঘ) Pod-এর env-এ লিখে

85. Managed ক্লাস্টারে node pool আলাদা করার সুবিধা কী? [Intermediate]

1. ক) খরচ বাড়ানো
2. খ) বিভিন্ন workload-এর জন্য আলাদা আকার/ক্ষমতার নোড দেওয়া ও আলাদাভাবে স্কেল করা
3. গ) DNS বন্ধ করা
4. ঘ) etcd মুছে ফেলা

মিশ্র ও প্রয়োগমূলক প্রশ্ন (অধ্যায় ১–২৪)

86. TechNova-র ওয়েব অ্যাপে হঠাৎ ট্রাফিক তিনগুণ বেড়েছে। প্রথমে কী করা যুক্তিযুক্ত? [Intermediate]

1. ক) HPA ও resource request যাচাই করে replica বাড়ানো
2. খ) etcd মুছে ফেলা
3. গ) Ingress মুছে ফেলা
4. ঘ) সব Pod-কে privileged করা

87. একটি Pod বারবার restart হচ্ছে, কিন্তু `kubectl logs` খালি। পরের ধাপ কী? [Intermediate]

1. ক) `kubectl logs --previous` ও `describe` দিয়ে আগের কনটেইনারের লগ ও event দেখা
2. খ) Service মুছে ফেলা
3. গ) নোড reboot করা
4. ঘ) Ingress বদলানো

88. Deployment-এর নতুন সংস্করণে ভুল ইমেজ গেছে — দ্রুততম নিরাপদ পদক্ষেপ কোনটি? [Beginner]

1. ক) `kubectl rollout undo deployment/technova-app`
2. খ) সব Pod মুছে ফেলা
3. গ) ক্লাস্টার মুছে ফেলা
4. ঘ) etcd restore করা

89. দুটি Pod একে অপরের সঙ্গে কথা বলতে পারছে না, কিন্তু একই Pod-এর ভেতরে curl কাজ করে। সম্ভাব্য কারণ কী? [Advanced]

1. ক) NetworkPolicy বা Service selector/DNS সমস্যা
2. খ) ইমেজের আকার
3. গ) HPA
4. ঘ) StorageClass

90. `kubectl get pods`-এ Pod `Running`, কিন্তু Service-এ ট্রাফিক যাচ্ছে না। প্রথমে কী যাচাই করবেন? [Intermediate]

1. ক) `kubectl get endpoints` — Service-এর selector Pod-এর label-এর সঙ্গে মিলছে কি না
2. খ) etcd-এর আকার
3. গ) Helm chart-এর নাম
4. ঘ) নোডের disk

91. StatefulSet-এর Pod-গুলো `pod-0, pod-1` নামে কেন আসে? [Intermediate]

1. ক) এলোমেলো নাম
2. খ) স্থিতিশীল, ক্রমিক পরিচয় — প্রতিটি Pod-এর নিজস্ব নাম, storage ও DNS থাকে
3. গ) Deployment-এর মতোই
4. ঘ) DaemonSet-এর নিয়ম

92. কোনটি stateless অ্যাপের জন্য সবচেয়ে উপযুক্ত workload? [Beginner]

1. ক) StatefulSet
2. খ) Deployment
3. গ) DaemonSet
4. ঘ) CronJob

93. PodDisruptionBudget (PDB) কী রক্ষা করে? [Advanced]

1. ক) Pod-এর CPU
2. খ) স্বেচ্ছায় বন্ধ (drain/upgrade) করার সময় ন্যূনতম কতগুলো Pod চলমান থাকতে হবে
3. গ) Pod-এর IP
4. ঘ) Pod-এর লগ

94. `kubectl drain` করার আগে কী করা উচিত? [Intermediate]

1. ক) কিছুই না
2. খ) নোড cordon করা আছে কি না যাচাই ও PDB/important workload সম্পর্কে সচেতন থাকা
3. গ) etcd মুছে ফেলা
4. ঘ) Ingress মুছে ফেলা

95. কোনটি Pod-এর নিরাপত্তা বাড়ায় না? [Intermediate]

1. ক) read-only root filesystem
2. খ) non-root ব্যবহারকারী
3. গ) privileged: true দেওয়া
4. ঘ) capability বাদ দেওয়া

96. Ingress-এ 502 Bad Gateway পেলে সম্ভাব্য কারণ কী? [Intermediate]

1. ক) Pod চলছে না বা Service-এর পোর্ট ভুল
2. খ) etcd বন্ধ
3. গ) Helm chart নেই
4. ঘ) ConfigMap নেই

97. ConfigMap-এর মান বদলে Deployment-এ rollout করাতে চাইলে কী করা হয়? [Advanced]

1. ক) কিছুই না — স্বয়ংক্রিয় হয়
2. খ) ConfigMap-এর checksum annotation হিসেবে যোগ করা বা Pod template-এ পরিবর্তন আনা
3. গ) Service মুছে ফেলা
4. ঘ) নোড reboot করা

98. কোনটি Kubernetes-এর declarative পদ্ধতির উদাহরণ? [Beginner]

1. ক) `kubectl run` দিয়ে হাতে Pod চালানো
2. খ) YAML-এ desired state লেখা ও `kubectl apply` দিয়ে ক্লাস্টারকে সেই অবস্থায় আনা
3. গ) SSH দিয়ে নোডে ঢুকে কনটেইনার চালানো
4. ঘ) হাতে হাতে Pod মুছে ফেলা

99. `kubectl apply` ও `kubectl create`-এর পার্থক্য কী? [Intermediate]

1. ক) কোনো পার্থক্য নেই
2. খ) apply declarative — আগের অবস্থা মনে রেখে পরিবর্তন প্রয়োগ করে; create নতুন resource তৈরি করে, থাকলে ব্যর্থ হয়
3. গ) create পুরনো resource বদলায়
4. ঘ) apply শুধু Pod-এ কাজ করে

100. একটি অ্যাপের ডেটাবেস Pod-এর ডেটা Pod মুছে গেলেও থাকতে হলে কী দরকার? [Beginner]

1. ক) emptyDir
2. খ) PersistentVolumeClaim দিয়ে স্থায়ী storage
3. গ) ConfigMap
4. ঘ) Secret

101. emptyDir কখন উপযুক্ত? [Intermediate]

1. ক) স্থায়ী ডেটাবেসে
2. খ) Pod-এর জীবদ্দশায় অস্থায়ী ভাগ করা ফাইল (যেমন cache) রাখতে
3. গ) ব্যাকআপে
4. ঘ) Secret-এ

102. কোনটি multi-container Pod-এর যুক্তিযুক্ত উদাহরণ? [Intermediate]

1. ক) দুটি আলাদা ওয়েব অ্যাপ এক Pod-এ
2. খ) মূল অ্যাপ + sidecar লগ/প্রক্সি কনটেইনার, যারা একই জীবনচক্র ভাগ করে
3. গ) ডেটাবেস + ওয়েব সার্ভার সবসময় এক Pod-এ
4. ঘ) দুটি আলাদা নোডের কাজ

103. Taint ও toleration কী করে? [Advanced]

1. ক) Pod-কে নোড থেকে সরায়
2. খ) Taint নোডকে Pod গ্রহণ থেকে বিরত রাখে, toleration থাকা Pod-ই শুধু সেখানে বসতে পারে
3. গ) Pod-এর সংখ্যা বাড়ায়
4. ঘ) DNS বদলায়

104. Node affinity ও nodeSelector-এর পার্থক্য কী? [Advanced]

1. ক) কোনো পার্থক্য নেই
2. খ) nodeSelector সরল label মিল; affinity নরম/কঠিন শর্ত, ওজন ও একাধিক নিয়ম দেয়
3. গ) affinity শুধু Pod-এর জন্য
4. ঘ) nodeSelector শুধু Service-এর জন্য

105. কোনটি Vertical Pod Autoscaler (VPA)-এর কাজ? [Intermediate]

1. ক) Pod-এর সংখ্যা বাড়ানো
2. খ) Pod-এর CPU/memory request/limit সমন্বয় করা
3. গ) নোডের সংখ্যা বাড়ানো
4. ঘ) Ingress রাউট করা

106. Cluster Autoscaler কী করে? [Intermediate]

1. ক) Pod-এর সংখ্যা বাড়ায়
2. খ) Pending Pod-এর জন্য নোড যোগ করে, খালি নোড সরিয়ে দেয়
3. গ) Service-এর IP বদলায়
4. ঘ) etcd ব্যাকআপ নেয়

107. কোনটি GitOps workflow-এর সঙ্গে সবচেয়ে সামঞ্জস্যপূর্ণ? [Intermediate]

1. ক) `kubectl edit deployment` দিয়ে সরাসরি বদলানো
2. খ) Git-এ pull request, review, merge — তারপর agent ক্লাস্টার মিলিয়ে নেয়
3. গ) SSH দিয়ে নোডে বদলানো
4. ঘ) হাতে হাতে Pod মুছে ফেলা

108. Helm values override করার নিরাপদ উপায় কোনটি? [Intermediate]

1. ক) chart-এর template সরাসরি বদলে ফেলা

2. খ) `--values` ফাইল বা `--set` দিয়ে, মূল chart না বদলে
3. গ) values.yaml মুছে ফেলা
4. ঘ) release মুছে ফেলা

109. Kustomize overlay-এর উদাহরণ কোনটি? [Intermediate]

1. ক) base-এ 2 replica, prod overlay-এ 5 replica ও আলাদা image tag
2. খ) base মুছে ফেলা
3. গ) chart তৈরি করা
4. ঘ) Secret Git-এ রাখা

110. কোনটি প্রোডাকশন Deployment-এর ভালো অভ্যাস? [Beginner]

1. ক) `:latest` ট্যাগ ব্যবহার করা
2. খ) নির্দিষ্ট image tag, resource request/limit, probe ও PodDisruptionBudget দেওয়া
3. গ) কোনো probe না দেওয়া
4. ঘ) সব Pod-কে privileged করা

111. Pod-এর `terminationGracePeriodSeconds` কী নিয়ন্ত্রণ করে? [Advanced]

1. ক) Pod শুরুর সময়
2. খ) SIGTERM পাঠানোর পর Pod-কে সুন্দরভাবে বন্ধ হওয়ার জন্য দেওয়া সময়
3. গ) Pod-এর CPU
4. ঘ) Pod-এর IP

112. PreStop hook কী কাজে লাগে? [Advanced]

1. ক) Pod শুরুর আগে কাজ করা
2. খ) Pod বন্ধ হওয়ার আগে সংযোগ শেষ করা বা cleanup করা
3. গ) Pod-এর লগ দেখা
4. ঘ) Pod-এর সংখ্যা বাড়ানো

113. কোনটি Service-এর ধরন নয়? [Beginner]

1. ক) ClusterIP
2. খ) NodePort
3. গ) LoadBalancer
4. ঘ) StatefulSet

114. LoadBalancer Service ক্লাউডে কী করে? [Beginner]

1. ক) Pod তৈরি করে
2. খ) ক্লাউডের load balancer তৈরি করে বাইরের IP দেয়
3. গ) DNS মুছে ফেলে

4. ঘ) etcd ব্যাকআপ নেয়

115. কোনটি RBAC-এর ভুল ব্যবহার? [Intermediate]

1. ক) namespace-ভিত্তিক Role দেওয়া
2. খ) সবাইকে cluster-admin দেওয়া
3. গ) ServiceAccount-কে সীমিত অনুমতি দেওয়া
4. ঘ) `can-i` দিয়ে যাচাই করা

116. একটি নোড `NotReady` হলে প্রথমে কী যাচাই করবেন? [Intermediate]

1. ক) `kubectl describe node` ও kubelet/container runtime-এর অবস্থা
2. খ) Ingress rule
3. গ) Helm chart
4. ঘ) ConfigMap

117. কোনটি etcd-এর জন্য সবচেয়ে ঝুঁকিপূর্ণ? [Advanced]

1. ক) নিয়মিত snapshot নেওয়া
2. খ) disk পূর্ণ হয়ে যাওয়া বা quorum হারানো
3. গ) Pod-এর সংখ্যা বাড়ানো
4. ঘ) Service তৈরি করা

118. কোনটি Kubernetes-এ secret ব্যবস্থাপনার ভালো পদ্ধতি নয়? [Beginner]

1. ক) External Secrets Operator ব্যবহার করা
2. খ) Secret-এর মান Git-এ plain text-এ রাখা
3. গ) RBAC দিয়ে পড়া সীমিত করা
4. ঘ) encryption at rest চালু করা

119. TechNova-র অ্যাপকে শূন্য downtime-এ আপডেট করতে কোন কৌশলটি Kubernetes Deployment-এর সঙ্গে সবচেয়ে স্বাভাবিক? [Intermediate]

1. ক) Recreate — সব Pod একসঙ্গে বন্ধ করে নতুন চালানো
2. খ) Rolling update — ধীরে ধীরে পুরনো Pod বদলে নতুন Pod আনা, readiness probe-সহ
3. গ) সব Pod মুছে ফেলা
4. ঘ) ক্লাস্টার মুছে নতুন তৈরি করা

120. একজন নতুন ইঞ্জিনিয়ারকে Kubernetes শেখার সঠিক ক্রম কোনটি? [Beginner]

1. ক) প্রথমে Helm, তারপর Pod
2. খ) Pod → Deployment/Service → ConfigMap/Secret → Ingress → RBAC/নিরাপত্তা → Helm/GitOps
3. গ) প্রথমে etcd, তারপর Pod
4. ঘ) প্রথমে GitOps, তারপর Pod

সংক্ষিপ্ত প্রশ্ন — ৩০টি

প্রতিটি প্রশ্নের উত্তর ৩–৬ বাক্যে নিজের ভাষায় লিখুন। উত্তর নির্দেশিকায় মূল পয়েন্টগুলো দেওয়া আছে।

1. Control plane ও worker node-এর ভূমিকার পার্থক্য ব্যাখ্যা করুন। [Beginner]

2. etcd কেন Kubernetes-এর জন্য এত গুরুত্বপূর্ণ? Quorum হারালে কী হয়? [Intermediate]

3. Pod ও কনটেইনারের পার্থক্য কী? কেন Pod-কে সবচেয়ে ছোট একক বলা হয়? [Beginner]

4. Deployment কীভাবে rolling update ও rollback করে? ReplicaSet-এর ভূমিকা কী? [Intermediate]

5. StatefulSet কখন ব্যবহার করবেন, উদাহরণসহ লিখুন। [Intermediate]

6. DaemonSet ও Deployment-এর ব্যবহারের পার্থক্য লিখুন। [Beginner]

7. Label, selector ও annotation-এর কাজ আলাদা করে ব্যাখ্যা করুন। [Intermediate]

8. Namespace ব্যবহারের তিনটি ব্যবহারিক সুবিধা লিখুন। [Beginner]

9. ConfigMap ও Secret-এর পার্থক্য ও নিরাপত্তা সতর্কতা লিখুন। [Beginner]

10. Resource request ও limit না দিলে কী ঝুঁকি হয়? উদাহরণ দিন। [Intermediate]

11. HPA কীভাবে কাজ করে? Metrics-server না থাকলে কী হবে? [Intermediate]

12. CNI কী এবং Pod-এর IP কীভাবে পাওয়া যায়, সংক্ষেপে লিখুন। [Intermediate]

13. ClusterIP, NodePort ও LoadBalancer Service-এর পার্থক্য লিখুন। [Beginner]

14. CoreDNS কীভাবে Service discovery করে? একই namespace ও অন্য namespace-এর উদাহরণ দিন। [Intermediate]

15. Ingress ও Ingress controller-এর পার্থক্য কী? Ingress ছাড়া কি Service বাইরে থেকে পাওয়া যায়? [Intermediate]

16. PV, PVC ও StorageClass-এর সম্পর্ক একটি উদাহরণ দিয়ে ব্যাখ্যা করুন। [Intermediate]

17. RBAC-এ Role, ClusterRole, RoleBinding ও ClusterRoleBinding কখন কোনটি ব্যবহার করবেন? [Intermediate]

18. Least privilege নীতি অনুসরণ না করলে কী ঝুঁকি হয়? [Beginner]

19. Pod Security Standards-এর তিনটি প্রোফাইল (privileged, baseline, restricted) সংক্ষেপে লিখুন। [Advanced]

20. Liveness ও readiness probe-এর পার্থক্য ও ভুল কনফিগারেশনের পরিণতি লিখুন। [Intermediate]

21. Pod `CrashLoopBackOff` হলে ধাপে ধাপে কীভাবে তদন্ত করবেন? [Intermediate]

22. Prometheus, Grafana ও Alertmanager একসঙ্গে কীভাবে observability দেয়? [Intermediate]

23. Helm chart-এর গঠন (Chart.yaml, values.yaml, templates) ব্যাখ্যা করুন। [Beginner]

24. Helm ও Kustomize-এর দর্শনের পার্থক্য কী? কখন কোনটি বেছে নেবেন? [Advanced]

25. ক্লাস্টার আপগ্রেডের আগে কোন তিনটি যাচাই সবচেয়ে জরুরি? [Advanced]

26. etcd ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা কেন জরুরি? RTO/RPO-র সঙ্গে সম্পর্ক লিখুন। [Advanced]

27. Managed Kubernetes (EKS/AKS/GKE) ব্যবহারের সুবিধা ও সীমাবদ্ধতা লিখুন। [Intermediate]

28. GitOps কী? Pull-based GitOps-এর সুবিধা লিখুন। [Intermediate]

29. NetworkPolicy দিয়ে কীভাবে একটি ডেটাবেস Pod-কে শুধু অ্যাপ Pod থেকে পৌঁছানো যায়, তা ব্যাখ্যা করুন। [Advanced]

30. TechNova-র একটি stateless ওয়েব অ্যাপ প্রোডাকশনে নিতে আপনার চেকলিস্টে কোন পাঁচটি জিনিস থাকবে? [Advanced]

ট্রাবলশুটিং সিনারিও — ২০টি

প্রতিটি সিনারিওতে পরিস্থিতি, লক্ষণ ও প্রশ্ন দেওয়া আছে। কী ভুল হয়েছে, কীভাবে নির্ণয় করবেন এবং কীভাবে ঠিক করবেন — তা লিখুন। উত্তর নির্দেশিকা অংশে বিস্তারিত সমাধান আছে।

সিনারিও 1 — Pod Pending [Beginner]

পরিস্থিতি: TechNova-র নতুন Deployment-এর Pod `Pending` অবস্থায় ২০ মিনিট ধরে আটকে আছে।

লক্ষণ: `kubectl describe pod`-এ `FailedScheduling: 0/2 nodes are available: 2 Insufficient cpu`।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 2 — ImagePullBackOff [Beginner]

পরিস্থিতি: নতুন ইমেজ ট্যাগ দিয়ে Deployment আপডেট করা হয়েছে।

লক্ষণ: Pod `ImagePullBackOff`; event-এ `Failed to pull image "technova/app:v2.1": not found`।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 3 — CrashLoopBackOff [Intermediate]

পরিস্থিতি: অ্যাপ Pod বারবার restart হচ্ছে।

লক্ষণ: `kubectl logs`-এ `Error: connect ECONNREFUSED 10.0.0.5:5432` (ডেটাবেস)।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 4 — OOMKilled [Intermediate]

পরিস্থিতি: ব্যস্ত সময়ে অ্যাপ Pod মাঝে মাঝে মারা যাচ্ছে।

লক্ষণ: `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 5 — Service-এ কোনো Endpoint নেই [Intermediate]

পরিস্থিতি: Pod `Running`, কিন্তু Service দিয়ে অ্যাপ পাওয়া যাচ্ছে না।

লক্ষণ: `kubectl get endpoints technova-app` খালি।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 6 — DNS ব্যর্থতা [Intermediate]

পরিস্থিতি: Pod থেকে `curl http://technova-api:8080` ব্যর্থ, কিন্তু IP দিয়ে কাজ করে।

লক্ষণ: `nslookup technova-api` ব্যর্থ; CoreDNS Pod চলছে।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 7 — Ingress 404 [Intermediate]

পরিস্থিতি: `https://app.technova.example/api` খুললে 404 আসে, সরাসরি Service-এ কাজ করে।

লক্ষণ: Ingress resource আছে, controller চলছে।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 8 — Ingress 502 [Intermediate]

পরিস্থিতি: Ingress দিয়ে অ্যাপ খুললে 502 Bad Gateway।

লক্ষণ: Service-এর পোর্ট 8080, কিন্তু Ingress-এ 80 লেখা; Pod readiness probe ব্যর্থ হচ্ছে।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 9 — PVC Pending [Intermediate]

পরিস্থিতি: ডেটাবেস StatefulSet-এর Pod শুরু হচ্ছে না।

লক্ষণ: PVC `Pending`; event-এ `no persistent volumes available` / provisioner error।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 10 — Node NotReady [Intermediate]

পরিস্থিতি: একটি worker নোড `NotReady` হয়ে গেছে, তার Pod-গুলো অন্য নোডে যাচ্ছে না।

লক্ষণ: `kubectl describe node`-এ kubelet ও container runtime সংক্রান্ত বার্তা, disk pressure সতর্কতা।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 11 — Rollout আটকে গেছে [Intermediate]

পরিস্থিতি: নতুন সংস্করণ deploy করার পর পুরনো Pod-গুলোই চলছে, নতুন Pod `Running` হচ্ছে না।

লক্ষণ: `kubectl rollout status` আটকে আছে; নতুন Pod-এ readiness probe ব্যর্থ।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 12 — RBAC: Forbidden [Intermediate]

পরিস্থিতি: CI Job থেকে `kubectl apply` করলে `Forbidden: User "system:serviceaccount:ci:deployer" cannot create deployments`।

লক্ষণ: ServiceAccount আছে, RoleBinding নেই।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 13 — ConfigMap বদলালেও অ্যাপ বদলায়নি [Advanced]

পরিস্থিতি: ConfigMap-এর মান বদলানো হয়েছে, কিন্তু অ্যাপ পুরনো মানই ব্যবহার করছে।

লক্ষণ: ConfigMap env var হিসেবে Pod-এ দেওয়া।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 14 — HPA কাজ করছে না [Intermediate]

পরিস্থিতি: CPU ৯০% হলেও replica বাড়ছে না।

লক্ষণ: `kubectl get hpa` -এ `<unknown>/80%`; `kubectl top pod` ব্যর্থ।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 15 — NetworkPolicy-তে অ্যাপ-ডেটাবেস সংযোগ বন্ধ [Advanced]

পরিস্থিতি: নতুন NetworkPolicy প্রয়োগের পর অ্যাপ ডেটাবেসে পৌঁছাতে পারছে না।

লক্ষণ: ডেটাবেস Pod-এ ingress শুধু একটি label-এর Pod থেকে অনুমোদিত, অ্যাপের label মিলছে না।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 16 — Secret ভুলভাবে mount [Intermediate]

পরিস্থিতি: অ্যাপ শুরু হচ্ছে না, লগে `DB_PASSWORD is empty`।

লক্ষণ: Secret-এর key `password`, কিন্তু Deployment-এ `DB_PASSWORD` হিসেবে `db-password` key খোঁজা হচ্ছে।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 17 — StatefulSet-এর Pod ক্রম ভেঙে গেছে [Advanced]

পরিস্থিতি: ডেটাবেসের একটি Pod মুছে যাওয়ার পর নতুন Pod পুরনো ডেটা পাচ্ছে না।

লক্ষণ: volumeClaimTemplates নেই; Pod emptyDir ব্যবহার করছে।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 18 — etcd disk পূর্ণ [Advanced]

পরিস্থিতি: নতুন resource তৈরি করা যাচ্ছে না, API server ধীর।

লক্ষণ: etcd-এর disk ৯৫% পূর্ণ; alarm: `etcd database space exceeded`।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 19 — Helm rollback ব্যর্থ [Advanced]

পরিস্থিতি: `helm rollback` করার পরও পুরনো সংস্করণ ফিরছে না।

লক্ষণ: `helm history` -এ revision আছে, কিন্তু একটি resource হাতে বদলানো হয়েছিল (`kubectl edit`)।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

সিনারিও 20 — GitOps OutOfSync [Advanced]

পরিস্থিতি: Argo CD-তে অ্যাপ `OutOfSync` ও `Degraded` দেখাচ্ছে।

লক্ষণ: কেউ `kubectl scale` দিয়ে replica হাতে বাড়িয়েছিল; Git-এ replica ৩ লেখা।

প্রশ্ন: কী ভুল হয়েছে এবং কীভাবে ঠিক করবেন?

প্রোডাকশন কেস স্টাডি — ১০টি

প্রতিটি কেস TechNova-র কাল্পনিক প্রোডাকশন ঘটনা। ঘটনা পড়ুন, তদন্তের প্রশ্নগুলোর উত্তর দিন, তারপর উত্তর নির্দেশিকার শিক্ষাগুলোর সঙ্গে মিলিয়ে দেখুন।

কেস 1 — ভুল ইমেজ ট্যাগে রাতের deploy [Intermediate]

ঘটনা: শুক্রবার রাত ২টায় TechNova-র API-এর নতুন সংস্করণ deploy করা হয়। CI-তে `:latest` ট্যাগ ব্যবহার করা হয়েছিল। নতুন Pod-গুলো crash করতে থাকে, কিন্তু পুরনো Pod-গুলোও ততক্ষণে সরিয়ে ফেলা হয়েছে। সকাল ৯টা পর্যন্ত API বন্ধ ছিল।

প্রভাব: ৭ ঘণ্টা downtime; গ্রাহকদের লগইন ও অর্ডার বন্ধ।

তদন্তের প্রশ্ন: কেন rollback দ্রুত করা যায়নি? `:latest` এখানে কী ঝুঁকি তৈরি করেছিল? কোন প্রক্রিয়া থাকলে ঘটনাটি এড়ানো যেত?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 2 — Resource limit না দেওয়ার মূল্য [Intermediate]

ঘটনা: TechNova-র একটি রিপোর্টিং Job-এ কোনো memory limit ছিল না। একটি বড় রিপোর্ট চালানোর সময় Job-টি নোডের সব মেমরি খেয়ে ফেলে; একই নোডের API Pod-গুলো OOMKilled হয়ে যায়।

প্রভাব: API ৪০ মিনিট ধরে মাঝে মাঝে বন্ধ; গ্রাহকদের অভিযোগ।

তদন্তের প্রশ্ন: কোন Pod-কে দোষ দেওয়া উচিত — Job-কে, নাকি সীমা না দেওয়া নীতিকে? LimitRange ও ResourceQuota কীভাবে সাহায্য করত?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 3 — Secret Git-এ ফাঁস [Advanced]

ঘটনা: একজন নতুন ইঞ্জিনিয়ার ডেটাবেসের পাসওয়ার্ডসহ একটি Secret YAML Git repository-তে push করেন। দুই সপ্তাহ পর repository-টি ভুল করে public হয়ে যায়।

প্রভাব: সব credential বদলাতে হয়; ডেটাবেসের অ্যাক্সেস লগ পরীক্ষা করতে হয়।

তদন্তের প্রশ্ন: কীভাবে ফাঁস ধরা পড়ত আগে? এখন কী কী পদক্ষেপ নিতে হবে? ভবিষ্যতে কী নীতি দরকার?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 4 — etcd ব্যাকআপ ছাড়া আপগ্রেড [Advanced]

ঘটনা: TechNova control plane আপগ্রেডের সময় etcd-এর snapshot নেওয়া হয়নি। আপগ্রেডের মাঝে একটি নোড ব্যর্থ হয়; ক্লাস্টারের state নষ্ট হয়।

প্রভাব: ১২ ঘণ্টা ধরে ক্লাস্টার পুনর্গঠন; কিছু Deployment হাতে আবার তৈরি করতে হয়।

তদন্তের প্রশ্ন: আপগ্রেডের আগে কোন ধাপগুলো বাদ পড়েছিল? পুনরুদ্ধার পরীক্ষা কেন জরুরি?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 5 — Ingress-এর একটিমাত্র ভুল পোর্ট [Beginner]

ঘটনা: নতুন সার্ভিসের Ingress-এ পোর্ট 80 লেখা হয়, কিন্তু অ্যাপ 8080-এ শোনে। বাইরে থেকে সব অনুরোধে 502 আসে; ভেতরে curl করলে অ্যাপ ঠিকঠাক কাজ করে।

প্রভাব: নতুন ফিচার লঞ্চের দিন ৩ ঘণ্টা ধরে গ্রাহকরা সার্ভিস পাননি।

তদন্তের প্রশ্ন: কেন ভেতরের পরীক্ষায় ধরা পড়েনি? কোন স্বয়ংক্রিয় যাচাই এটা ধরত?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 6 — HPA ছাড়া ফ্ল্যাশ সেল [Intermediate]

ঘটনা: TechNova-র ফ্ল্যাশ সেলে ট্রাফিক ১০ গুণ বাড়ে। HPA কনফিগার করা ছিল না; replica হাতে বাড়াতে বাড়াতে ২৫ মিনিট চলে যায়।

প্রভাব: সেলের প্রথম ২৫ মিনিটে অনেক অর্ডার ব্যর্থ; আয় ক্ষতি।

তদন্তের প্রশ্ন: HPA ছাড়াও কোন প্রস্তুতি দরকার ছিল (load test, PDB, resource)? কেন হাতে স্কেল করা যথেষ্ট নয়?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 7 — Cluster-admin সবার হাতে [Advanced]

ঘটনা: TechNova-র সব ডেভেলপারকে সুবিধার জন্য cluster-admin দেওয়া ছিল। একজন ভুল করে `kubectl delete namespace prod` চালান।

প্রভাব: প্রোডাকশন namespace মুছে যায়; ব্যাকআপ থেকে ফেরাতে ৫ ঘণ্টা লাগে।

তদন্তের প্রশ্ন: RBAC কীভাবে এটা আটকাত? namespace মুছে যাওয়া থেকে বাঁচার আর কী উপায় আছে?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 8 — PVC ভরে যাওয়া ডেটাবেস [Intermediate]

ঘটনা: TechNova-র PostgreSQL PVC ১০ Gi দিয়ে তৈরি করা হয়েছিল; কোনো disk alert ছিল না। একদিন disk ১০০% পূর্ণ হয়ে ডেটাবেস লেখা বন্ধ করে দেয়।

প্রভাব: ২ ঘণ্টা ধরে নতুন অর্ডার নেওয়া যায়নি।

তদন্তের প্রশ্ন: কোন monitoring/alert থাকলে আগে ধরা পড়ত? PVC বাড়ানো (expand) কীভাবে করা যায়?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 9 — GitOps-এর বাইরে হাতে বদল [Advanced]

ঘটনা: একজন ইঞ্জিনিয়ার জরুরি প্রয়োজনে `kubectl scale` দিয়ে replica ১০ করেন, Git-এ বদলাননি। রাতে Argo CD sync করে replica আবার ৩-এ ফিরিয়ে আনে; ট্রাফিকের চাপে অ্যাপ ধীর হয়ে যায়।

প্রভাব: রাতের ব্যস্ত সময়ে latency বেড়ে যায়।

তদন্তের প্রশ্ন: কেন দুটি সত্যের উৎস (Git ও হাতে বদল) একসঙ্গে চলতে পারে না? জরুরি বদলের সঠিক পদ্ধতি কী?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

কেস 10 — নোড drain না করে রিবুট [Intermediate]

ঘটনা: একজন অপারেটর একটি worker নোড drain না করেই reboot করেন। ওই নোডের একমাত্র replica-ওয়ালা একটি গুরুত্বপূর্ণ Pod ১৫ মিনিট বন্ধ থাকে।

প্রভাব: একটি অভ্যন্তরীণ সার্ভিস ১৫ মিনিট অনুপলব্ধ।

তদন্তের প্রশ্ন: drain কী করে? PDB ও একাধিক replica কীভাবে এটা কমাত? নোড রক্ষণাবেক্ষণের সঠিক ক্রম কী?

শিক্ষা: উত্তর নির্দেশিকায় দেখুন।

ফাইনাল পরীক্ষা — ব্যবহারিক মূল্যায়ন

এই পরীক্ষাটি kind বা Minikube-এ একটি পরীক্ষামূলক ক্লাস্টারে দিতে হবে। প্রোডাকশন ক্লাস্টার ব্যবহার করবেন না। পরীক্ষার আগে ক্লাস্টারটি নতুন করে তৈরি করুন এবং সব কাজ `technova-exam` namespace-এ করুন।

সময়সূচি — মোট ৩ ঘণ্টা

অংশ	সময়	নম্বর	ধরন
A — তত্ত্ব (MCQ + সংক্ষিপ্ত)	৪৫ মিনিট	৩০	লিখিত
B — ক্লাস্টার সেটআপ ও workload deploy	৪৫ মিনিট	৩০	হাতে-কলমে
C — Networking, storage ও নিরাপত্তা	৪৫ মিনিট	২৫	হাতে-কলমে
D — ট্রাবলশুটিং ও rollback	৩০ মিনিট	১৫	হাতে-কলমে
মোট	৩ ঘণ্টা (বিরতি ছাড়া)	১০০	—

পরীক্ষার সময় ইন্টারনেট ও বইয়ের রেফারেন্স অংশ দেখা যাবে; অন্যের সঙ্গে কথা বলা বা অন্যের ক্লাস্টার ব্যবহার করা যাবে না।

অংশ A — তত্ত্ব (৩০ নম্বর)

- A1 (১০ নম্বর): এই বইয়ের MCQ থেকে ১০টি প্রশ্ন — প্রতিটি ১ নম্বর।
- A2 (১০ নম্বর): সংক্ষিপ্ত প্রশ্ন থেকে ২টি — প্রতিটি ৫ নম্বর (control plane; Service/DNS)।
- A3 (১০ নম্বর): একটি কেস স্টাডি পড়ে কারণ, প্রভাব ও প্রতিরোধের উপায় লিখুন।

অংশ B — ক্লাস্টার সেটআপ ও workload (৩০ নম্বর)

- B1 (৫): `technova-exam` namespace তৈরি করুন; `kubectl cluster-info` ও `kubectl get nodes` দিয়ে ক্লাস্টারের অবস্থা যাচাই করুন।
- B2 (১০): TechNova ওয়েব অ্যাপের Deployment তৈরি করুন — image `nginx:1.27` (উদাহরণ হিসেবে), ৩টি replica, resource request/limit, liveness ও readiness probe, label `app=technova-web`।
- B3 (৫): ConfigMap থেকে একটি env var ও Secret থেকে একটি env var Pod-এ দিন (মান: `CHANGE-ME` placeholder, বাস্তব secret নয়)।
- B4 (৫): Deployment-কে নতুন image tag-এ rolling update করুন; `kubectl rollout status` দিয়ে যাচাই করুন।
- B5 (৫): HPA তৈরি করুন — CPU ৫০% লক্ষ্য, min ২, max ৬ replica।

অংশ C — Networking, storage ও নিরাপত্তা (২৫ নম্বর)

- C1 (৫): ClusterIP Service তৈরি করুন; একই namespace-এর একটি অস্থায়ী Pod থেকে `curl` করে Service-এর নামে অ্যাপ পৌঁছানো যায় তা দেখান।

- C2 (৫): Ingress তৈরি করুন — host `app.technova.example`, path `/` → Service; controller ইনস্টল থাকলে `curl -H "Host: app.technova.example"` দিয়ে যাচাই করুন।
- C3 (৫): ১ Gi PVC (StorageClass: পরীক্ষামূলক ক্লাস্টারের ডিফল্ট) তৈরি করে একটি Pod-এ `/data`-তে mount করুন; ফাইল লিখে Pod মুছে আবার তৈরি করলে ফাইল থাকে কি না যাচাই করুন।
- C4 (৫): একটি NetworkPolicy লিখুন যা শুধু `app=technova-web` Pod থেকে ডেটাবেস Pod-এর 5432 পোর্টে যাওয়া অনুমোদন করে।
- C5 (৫): একটি Role ও RoleBinding তৈরি করুন যা `exam-user` ServiceAccount-কে শুধু `technova-exam` namespace-এ Pod পড়তে দেয়; `kubectl auth can-i` দিয়ে যাচাই করুন।

অংশ D — ট্রাবলশুটিং ও rollback (১৫ নম্বর)

- D1 (৫): পরীক্ষক একটি ভুল ইমেজ ট্যাগ দিয়ে Deployment আপডেট করবেন — আপনি কারণ নির্ণয় করে (`describe` / `logs`) লিখুন।
- D2 (৫): `kubectl rollout undo` দিয়ে আগের সংস্করণে ফিরে যান এবং যাচাই করুন।
- D3 (৫): একটি Pod-এর লগ ও event থেকে সমস্যার মূল কারণ এক প্যারাগ্রাফে লিখুন।

নম্বর দেওয়ার নিয়ম (Marking Scheme)

অংশ	পূর্ণ নম্বর	পাসের জন্য ন্যূনতম
A — তত্ত্ব	৩০	১৫
B — Deploy	৩০	১৮
C — Networking/Storage/নিরাপত্তা	২৫	১৩
D — ট্রাবলশুটিং	১৫	৮
মোট	**১০০**	**৬০**

পাসের মানদণ্ড

- মোট ৬০% বা তার বেশি এবং প্রতিটি অংশে ন্যূনতম নম্বর পেতে হবে।
- কোনো অংশে শূন্য পেলে সামগ্রিকভাবে ফেল বলে ধরা হবে, মোট ৬০ হলেও।
- গ্রেড: ৯০+ = Distinction; ৭৫–৮৯ = Merit; ৬০–৭৪ = Pass; ৬০-এর নিচে = পুনরায় পরীক্ষা।
- পরীক্ষক YAML-এর সঠিকতা, `kubectl get/describe` আউটপুট ও লিখিত ব্যাখ্যা — তিনটিই যাচাই করবেন। শুধু কমান্ড মুখস্থ করলে পূর্ণ নম্বর মিলবে না; কেন করছেন, তা ব্যাখ্যা করতে হবে।

> **পরীক্ষার পর:** পরীক্ষামূলক ক্লাস্টারটি মুছে ফেলুন (`kind delete cluster` বা `minikube delete`)। কোনো Secret বা kubeconfig পরীক্ষার পরে ফেলে রাখবেন না।

উত্তর নির্দেশিকা — MCQ ১–৬০ ও সংক্ষিপ্ত প্রশ্ন ১–১৫

প্রতিটি উত্তরের পাশে সংক্ষিপ্ত ব্যাখ্যা দেওয়া আছে। শুধু অক্ষরটি না দেখে ব্যাখ্যাটি পড়ুন।

MCQ ১–৬০: উত্তর ও ব্যাখ্যা

প্রশ্ন	উত্তর	ব্যাখ্যা
1	খ) etcd	etcd-ই ক্লাস্টারের সব desired ও actual state সংরক্ষণ করে; API server তার মাধ্যমেই কাজ করে।
2	খ	API server সব অনুরোধের প্রবেশদ্বার — authentication, authorization, admission, তারপর etcd-তে লেখা।
3	খ	Scheduler প্রথমে filter করে (resource, taint, affinity), তারপর score করে সেরা নোড বেছে নেয়।
4	খ	Quorum হারালে etcd লেখা বন্ধ করে দেয়; চলমান Pod চলতে থাকে, কিন্তু নতুন পরিবর্তন করা যায় না।
5	ঘ	kube-controller-manager control plane-এর অংশ; worker-এ kubelet, kube-proxy ও runtime থাকে।
6	ক	Pod-ই সবচেয়ে ছোট deployable একক; একা কনটেইনার সরাসরি deploy করা যায় না।
7	খ	একই Pod-এর কনটেইনার একই network namespace (একই IP) ও volume ভাগ করে।
8	খ	Init container মূল কনটেইনারের আগে চলে ও শেষ হলে মূল কনটেইনার শুরু হয়।
9	খ	Never নীতি একবার চলা কাজের জন্য; দীর্ঘমেয়াদি অ্যাপে Always দরকার।
10	খ	Pending মানে scheduler নোড পায়নি — resource, nodeSelector, taint বা affinity যাচাই করুন।
11	খ	Deployment ReplicaSet তৈরি করে; ReplicaSet Pod-এর replica সংখ্যা ধরে রাখে।
12	ক	maxUnavailable 0 মানে কোনো Pod অনুপলব্ধ হবে না; maxSurge 1 মানে একটি অতিরিক্ত Pod আগে তৈরি হবে।
13	খ	স্থিতিশীল পরিচয়, ক্রমিক চালু ও প্রতি Pod-এর নিজস্ব storage দরকার হলে StatefulSet।
14	খ	DaemonSet প্রতিটি (উপযুক্ত) নোডে একটি Pod চালায় — লগ/মনিটরিং এজেন্টের জন্য আদর্শ।
15	খ	Job একবার সম্পন্ন হওয়ার কাজ; CronJob সময়সূচি অনুযায়ী Job তৈরি করে।
16	খ	Namespace নাম ও নীতি অনুযায়ী রিসোর্স আলাদা করে — পরিবেশ, দল বা অ্যাপ অনুযায়ী।
17	খ	Label selector-এ ব্যবহৃত হয়; annotation শুধু তথ্য রাখে, selector-এ কাজ করে না।
18	খ	Service label selector মিলিয়ে Endpoints তৈরি করে; Pod বদলালেও Service স্থির থাকে।
19	খ	কমা দিয়ে দেওয়া একাধিক label-ই মিলতে হয় (AND)।
20	খ	ResourceQuota namespace স্তরে মোট ব্যবহার সীমিত করে।
21	খ	ConfigMap সংবেদনশীল নয় এমন কনফিগারেশনের জন্য; Secret সংবেদনশীল তথ্যের জন্য।

22	খ	Secret base64-encoded থাকে, ডিফল্টে encrypt নয় — encryption at rest আলাদাভাবে চালু করতে হয়।
23	ক	ConfigMap env var হিসেবে বা volume mount করে Pod-এ দেওয়া যায়।
24	খ	Volume mount-এ kubelet ফাইল আপডেট করে, কিন্তু অ্যাপকে reload করতে হয়; env var-এ Pod restart দরকার।
25	খ	RBAC দিয়ে পড়া সীমিত করা ও বাহ্যিক secret manager ব্যবহার করাই নিরাপদ পদ্ধতি।
26	খ	Requests scheduler-এর গ্যারান্টি; limits সর্বোচ্চ সীমা — CPU throttle, memory OOM হতে পারে।
27	খ	মেমরি limit ছাড়ালে কনটেইনার OOMKilled হয় (exit 137)।
28	খ	HPA metric-এর ভিত্তিতে replica সংখ্যা সমন্বয় করে।
29	ক	CPU/memory metric-এর জন্য metrics-server (বা সমতুল্য API) দরকার।
30	খ	সব কনটেইনারে CPU ও memory-র request = limit হলে QoS Guaranteed হয়।
31	ক	প্রতিটি Pod-এর নিজস্ব IP থাকে; সব Pod NAT ছাড়াই কথা বলতে পারে — এটাই Kubernetes মডেল।
32	খ	CNI Pod-এর নেটওয়ার্ক ইন্টারফেস ও IP বরাদ্দের প্লাগইন কাঠামো।
33	খ	kube-proxy Service-এর নিয়ম (iptables/IPVS) রক্ষণাবেক্ষণ করে ট্রাফিক Pod-এ পাঠায়।
34	খ	Service-এর স্থিতিশীল ClusterIP ও Endpoints Pod-এর পরিবর্তনশীল IP থেকে অ্যাপকে আলাদা রাখে।
35	ক	NetworkPolicy না থাকলে সব অনুমোদিত; Policy নির্বাচিত Pod-কে সীমিত করে।
36	খ	ClusterIP শুধু ক্লাস্টারের ভেতরের যোগাযোগের জন্য।
37	ক	NodePort প্রতিটি নোডে 30000–32767 পোর্ট খুলে বাইরে থেকে ট্রাফিক আনে।
38	খ	Headless Service প্রতিটি Pod-এর IP DNS-এ দেয় — StatefulSet-এর জন্য দরকারি।
39	খ	এটি default namespace-এর technova-api Service-এর সম্পূর্ণ DNS নাম।
40	ক	একই namespace-এ শুধু Service-এর নামেই ডাকা যায়।
41	খ	Ingress HTTP/HTTPS ট্রাফিক host/path অনুযায়ী Service-এ রাউট করে।
42	খ	Ingress resource একা কিছু করে না; controller-ই rule পড়ে ট্রাফিক রাউট করে।
43	ক	TLS-এর জন্য TLS Secret ও tls অংশে host দরকার।
44	খ	Gateway API ভূমিকা আলাদা করে, এক্সটেনশন ও ক্রস-namespace রাউটিং ভালোভাবে দেয়।
45	খ	404 মানে রাউটিং মিলছে না — host/path ও Service-এর নাম-পোর্ট যাচাই করুন।
46	খ	PV সরবরাহ, PVC চাহিদা; PVC উপযুক্ত PV-এর সঙ্গে bind হয়।
47	খ	StorageClass dynamic provisioning-এর নিয়ম দেয় — PVC এলেই PV তৈরি হয়।
48	খ	CSI বাইরের storage-কে Kubernetes-এর সঙ্গে যুক্ত করার মানসম্মত ইন্টারফেস।
49	খ	PVC Pending মানে উপযুক্ত StorageClass/PV নেই বা provisioner ব্যর্থ।

50	খ	ReadWriteOnce মানে একটিমাত্র নোড থেকে read-write mount।
51	খ	Role/RoleBinding namespace-এর মধ্যে; ClusterRole/ClusterRoleBinding ক্লাস্টারজুড়ে।
52	খ	ServiceAccount Pod/অ্যাপের পরিচয়; RBAC দিয়ে তাকে অনুমতি দেওয়া হয়।
53	খ	`can-i` বর্তমান ব্যবহারকারীর অনুমতি যাচাই করে, resource তৈরি করে না।
54	খ	Least privilege মানে যতটুকু দরকার, ঠিক ততটুকুই দেওয়া।
55	খ	Token না থাকলে চুরি/অপব্যবহারের ঝুঁকি কমে।
56	খ	Restricted প্রোফাইলে non-root, read-only filesystem, capability বাদ দেওয়া ইত্যাদি কঠোর নিয়ম থাকে।
57	ক	runAsNonRoot: true মানে Pod root হিসেবে চলবে না।
58	খ	allowPrivilegeEscalation: false প্রক্রিয়ার বেশি সুযোগ পাওয়ার পথ বন্ধ করে।
59	ক	:latest ট্যাগে Always নীতি দরকার, নইলে পুরনো cached ইমেজ চলতে পারে।
60	খ	Admission controller API server-এ কাজ করে — etcd-তে যাওয়ার আগে অনুরোধ যাচাই করে।

সংক্ষিপ্ত প্রশ্ন ১–১৫: মূল পয়েন্ট

1. **Control plane বনাম worker:** Control plane সিদ্ধান্ত নেয় (API server, etcd, scheduler, controller-manager); worker workload চালায় (kubelet, kube-proxy, runtime)।
2. **etcd:** সব state-এর উৎস; quorum হারালে লেখা বন্ধ, চলমান workload চলে; নিয়মিত snapshot দরকার।
3. **Pod বনাম কনটেইনার:** Pod একাধিক কনটেইনারের সমষ্টি, একই IP ও জীবনচক্র; কনটেইনার একা deploy হয় না।
4. **Deployment:** ReplicaSet-এর মাধ্যমে replica ধরে রাখে; rolling update-এ ধীরে ধীরে Pod বদলায়; পুরনো ReplicaSet রেখে rollback করে।
5. **StatefulSet:** ডেটাবেসের মতো workload — স্থিতিশীল নাম, ক্রমিক চালু, প্রতিটি Pod-এর নিজস্ব PVC।
6. **DaemonSet বনাম Deployment:** DaemonSet প্রতি নোডে একটি Pod; Deployment নির্দিষ্ট সংখ্যক replica যেকোনো নোডে।
7. **Label/selector/annotation:** Label পরিচয় ও নির্বাচন; selector label মেলায়; annotation শুধু তথ্য।
8. **Namespace:** পরিবেশ/দল অনুযায়ী বিভাজন, নামের সংঘাত এড়ানো, ResourceQuota/RBAC আলাদা করা।
9. **ConfigMap বনাম Secret:** ConfigMap সংবেদনশীল নয় এমন কনফিগ; Secret সংবেদনশীল তথ্য — base64, Git-এ রাখা যাবে না, RBAC ও encryption দরকার।
10. **Request/limit না দিলে:** একটি Pod সব resource খেয়ে ফেলতে পারে; scheduler ভুল সিদ্ধান্ত নেয়; OOM ও throttle-এর ঝুঁকি।
11. **HPA:** metric-এর ভিত্তিতে replica সমন্বয় করে; metrics-server ছাড়া CPU/memory metric পাওয়া যায় না।
12. **CNI:** Pod-এর নেটওয়ার্ক ইন্টারফেস ও IP বরাদ্দের প্লাগইন কাঠামো; প্রতিটি Pod নিজস্ব IP পায়।
13. **Service-এর ধরন:** ClusterIP ভেতরে; NodePort নোডের পোর্টে; LoadBalancer ক্লাউড load balancer দিয়ে বাইরে।
14. **CoreDNS:** Service-এর নামকে ClusterIP-তে অনুবাদ করে; একই namespace-এ শুধু নাম, অন্য namespace-এ নাম.namespace।

15. **Ingress বনাম controller:** Ingress শুধু rule; controller সেই rule পড়ে ট্রাফিক রাউট করে। NodePort/LoadBalancer দিয়েও বাইরে থেকে পাওয়া যায়, তবে host/path রাউটিং Ingress-এর কাজ।

উত্তর নির্দেশিকা (২য় অংশ) — MCQ ৬১–১২০, সংক্ষিপ্ত ১৬–৩০, সিনারিও, কেস স্টাডি, ফাইনাল পরীক্ষা

MCQ ৬১–১২০: উত্তর ও ব্যাখ্যা

প্রশ্ন	উত্তর	ব্যাখ্যা
61	খ	CrashLoopBackOff-এ logs ও describe-ই প্রথম ধাপ — কেন crash হচ্ছে তা জানা যায়।
62	খ	ImagePullBackOff মানে ইমেজ টানা যাচ্ছে না — নাম/ট্যাগ, registry বা credential যাচাই করুন।
63	খ	Liveness ব্যর্থ হলে restart; readiness ব্যর্থ হলে Endpoint থেকে সরানো হয়।
64	খ	Events-এ FailedScheduling, FailedMount, BackOff, Unhealthy-এর মতো কারণ লেখা থাকে।
65	খ	Prometheus /metrics endpoint থেকে pull করে; ServiceMonitor সেই লক্ষ্য নির্ধারণে সাহায্য করে।
66	খ	Observability-র তিন স্তম্ভ: logs, metrics, traces।
67	ক	Pod restart হলে বর্তমান লগ খালি হতে পারে; --previous দিয়ে আগের কনটেইনারের লগ দেখুন।
68	খ	kubectl top-এর জন্য metrics-server দরকার।
69	ক	rollout history-তে revision-এর ইতিহাস দেখা যায়।
70	খ	ব্যবহারকারীর ওপর প্রভাব ফেলে এমন লক্ষণে অল্প, কার্যকর alert দিন — alert fatigue এড়ান।
71	খ	Helm chart হলো manifest টেমপ্লেটের প্যাকেজ — Chart.yaml, values.yaml, templates/।
72	খ	upgrade --install idempotent — না থাকলে ইনস্টল, থাকলে আপগ্রেড।
73	খ	Revision দিয়ে helm rollback করে আগের সংস্করণে ফেরা যায়।
74	খ	Kustomize টেমপ্লেট ছাড়াই base-এর ওপর patch/overlay দিয়ে পরিবেশভেদে পরিবর্তন করে।
75	খ	kustomization.yaml-এ resources, images, labels, patches থাকে।
76	খ	আগে control plane (একটি minor version), তারপর worker নোড।
77	খ	etcdctl snapshot save দিয়ে snapshot নিন, নিরাপদে রাখুন, পুনরুদ্ধার পরীক্ষা করুন।
78	ক	RTO = কত দ্রুত ফিরতে হবে; RPO = কতটুকু ডেটা হারানো সহ্য করা যায়।
79	খ	Velero-জাতীয় টুল resource ও PV ডেটা ব্যাকআপ নেয়।
80	খ	আপগ্রেডের আগে deprecated/removed API version যাচাই করা সবচেয়ে জরুরি।
81	খ	Managed Kubernetes-এ প্রদানকারী control plane পরিচালনা করে।
82	খ	GitOps-এ Git-ই desired state-এর উৎস; agent ক্লাস্টারকে Git-এর সঙ্গে মিলিয়ে রাখে।
83	খ	OutOfSync মানে ক্লাস্টারের অবস্থা Git-এর সঙ্গে মিলছে না।

84	খ	Sealed Secrets বা External Secrets দিয়ে Secret নিরাপদে পরিচালনা করুন।
85	খ	Node pool আলাদা করলে বিভিন্ন workload-এর জন্য আলাদা আকারের নোড ও আলাদা স্কেল করা যায়।
86	ক	ট্রাফিক বাড়লে প্রথমে HPA ও resource যাচাই করে replica বাড়ান।
87	ক	লগ খালি হলে --previous ও describe দিয়ে আগের কনটেইনার ও event দেখুন।
88	ক	rollout undo দ্রুততম নিরাপদ rollback।
89	ক	Pod-এর ভেতরে কাজ করলে সমস্যা Pod-এর মধ্যে নয় — NetworkPolicy, Service selector বা DNS যাচাই করুন।
90	ক	Endpoints খালি মানে selector/label মিলছে না।
91	খ	StatefulSet স্থিতিশীল, ক্রমিক নাম দেয় — প্রতিটি Pod-এর নিজস্ব পরিচয় ও storage থাকে।
92	খ	Stateless অ্যাপের জন্য Deployment সবচেয়ে স্বাভাবিক।
93	খ	PDB স্বেচ্ছায় বন্ধের সময় ন্যূনতম চলমান Pod নিশ্চিত করে।
94	খ	Drain-এর আগে cordon ও PDB সম্পর্কে সচেতন থাকুন।
95	গ	privileged: true নিরাপত্তা কমায়, বাড়ায় না।
96	ক	502 মানে backend পৌঁছানো যাচ্ছে না — Pod চলছে কি না ও Service-এর পোর্ট যাচাই করুন।
97	খ	ConfigMap বদলালে Pod নিজে restart হয় না; checksum annotation বা template পরিবর্তন করে rollout করাতে হয়।
98	খ	YAML-এ desired state লেখা ও apply করা — এটাই declarative পদ্ধতি।
99	খ	apply declarative ও idempotent; create নতুন তৈরি করে, থাকলে ব্যর্থ হয়।
100	খ	স্থায়ী ডেটার জন্য PVC দরকার; emptyDir Pod মুছলে যায়।
101	খ	emptyDir Pod-এর জীবদ্দশায় অস্থায়ী ভাগ করা ফাইলের জন্য।
102	খ	Sidecar প্যাটার্ন — মূল অ্যাপ + লগ/প্রক্সি কনটেইনার একই জীবনচক্র ভাগ করে।
103	খ	Taint নোডকে Pod গ্রহণ থেকে বিরত রাখে; toleration থাকা Pod-ই সেখানে বসতে পারে।
104	খ	nodeSelector সরল label মিল; affinity নরম/কঠিন শর্ত ও ওজন দেয়।
105	খ	VPA Pod-এর CPU/memory request/limit সমন্বয় করে।
106	খ	Cluster Autoscaler Pending Pod-এর জন্য নোড যোগ করে, খালি নোড সরায়।
107	খ	GitOps-এ পরিবর্তন Git-এ pull request দিয়ে আসে; হাতে kubectl edit নয়।
108	খ	--values বা --set দিয়ে override করুন; মূল chart বদলাবেন না।
109	ক	Overlay-এ base-এর replica/image বদলানোই Kustomize-এর স্বাভাবিক ব্যবহার।
110	খ	নির্দিষ্ট tag, request/limit, probe ও PDB — প্রোডাকশনের ভালো অভ্যাস।

111	খ	terminationGracePeriodSeconds SIGTERM-এর পর সুন্দরভাবে বন্ধ হওয়ার সময় দেয়।
112	খ	PreStop hook Pod বন্ধ হওয়ার আগে cleanup/সংযোগ শেষ করতে দেয়।
113	ঘ	StatefulSet Service-এর ধরন নয়; এটি workload।
114	খ	LoadBalancer Service ক্লাউড load balancer তৈরি করে বাইরের IP দেয়।
115	খ	সবাইকে cluster-admin দেওয়া RBAC-এর ভুল ব্যবহার।
116	ক	NotReady-তে describe node ও kubelet/runtime-এর অবস্থা যাচাই করুন।
117	খ	etcd-এর disk পূর্ণ হওয়া বা quorum হারানো সবচেয়ে ঝুঁকিপূর্ণ।
118	খ	Secret Git-এ plain text-এ রাখা কখনোই নিরাপদ নয়।
119	খ	Rolling update + readiness probe শূন্য downtime-এর স্বাভাবিক উপায়।
120	খ	Pod → Deployment/Service → ConfigMap/Secret → Ingress → RBAC/নিরাপত্তা → Helm/GitOps — এটাই যুক্তিযুক্ত ক্রম।

সংক্ষিপ্ত প্রশ্ন ১৬–৩০: মূল পয়েন্ট

1. **PV/PVC/StorageClass:** PV সরবরাহ, PVC চাহিদা, StorageClass dynamic provisioning-এর নিয়ম; PVC তৈরি হলে উপযুক্ত PV bind হয়, Pod PVC mount করে।
2. **RBAC:** এক namespace-এ Role+RoleBinding; ক্লাস্টারজুড়ে ClusterRole+ClusterRoleBinding; ClusterRole-কে RoleBinding দিয়েও এক namespace-এ ব্যবহার করা যায়।
3. **Least privilege না মানলে:** ভুল কমান্ডে প্রোডাকশন মুছে যাওয়া, credential চুরি হলে পুরো ক্লাস্টার আপস — ঝুঁকি ছড়িয়ে পড়ে।
4. **Pod Security প্রোফাইল:** privileged — প্রায় সব অনুমতি; baseline — পরিচিত ঝুঁকিপূর্ণ বৈশিষ্ট্য বন্ধ; restricted — কঠোরতম (non-root, read-only, capability বাদ)।
5. **Probe:** liveness ব্যর্থ হলে restart, readiness ব্যর্থ হলে Endpoint থেকে সরানো; ভুল probe-এ সুস্থ Pod-ও restart/ট্রাফিক হারাতে পারে।
6. **CrashLoopBackOff তদন্ত:** describe → events, logs, logs --previous, env/Secret/ConfigMap যাচাই, image ও command যাচাই, resource যাচাই।
7. **Prometheus/Grafana/Alertmanager:** Prometheus metric সংগ্রহ করে, Grafana দেখায়, Alertmanager সতর্কবার্তা পাঠায়।
8. **Helm chart:** Chart.yaml (পরিচয়), values.yaml (ডিফল্ট মান), templates/ (manifest টেমপ্লেট) — একসঙ্গে প্যাকেজ।
9. **Helm বনাম Kustomize:** Helm টেমপ্লেট + প্যাকেজিং ও সংস্করণ; Kustomize টেমপ্লেট ছাড়া patch/overlay — সহজ, GitOps-বান্ধব।
10. **আপগ্রেডের আগে:** deprecated API যাচাই, etcd snapshot, ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা, staging-এ পরীক্ষা।
11. **ব্যাকআপ পরীক্ষা:** ব্যাকআপ না পরীক্ষা করলে পুনরুদ্ধারের সময় জানা যায় না তা কাজ করে কি না; RTO/RPO-র সঙ্গে মিলিয়ে পরীক্ষা করুন।

12. **Managed Kubernetes:** সুবিধা — control plane পরিচালনার ভার কমে; সীমা — খরচ, নিয়ন্ত্রণ কম, প্রদানকারীর ওপর নির্ভরতা।
13. **GitOps:** Git-ই desired state-এর উৎস; pull-based agent Git দেখে ক্লাস্টার মিলিয়ে রাখে — audit, review ও rollback সহজ।
14. **NetworkPolicy:** ডেটাবেস Pod-এ ingress শুধু app=technova-web label-এর Pod থেকে 5432 পোর্টে অনুমোদন করুন; বাকি সব বন্ধ।
15. **প্রোডাকশন চেকলিস্ট:** নির্দিষ্ট image tag, request/limit, probe, PDB, RBAC, Secret ব্যবস্থাপনা, Ingress/TLS, monitoring/alert, ব্যাকআপ — যেকোনো পাঁচটি যুক্তিযুক্ত।

সিনারিও ১–২০: সমাধান

1. **Pending — Insufficient CPU:** Resource request কমান, replica কমান বা নোড যোগ করুন; `kubectl describe pod` ও `kubectl top nodes` দিয়ে যাচাই করুন।
2. **ImagePullBackOff — not found:** ইমেজের নাম/ট্যাগ যাচাই করুন, registry-তে ইমেজ আছে কি না দেখুন, private registry হলে imagePullSecrets দিন।
3. **CrashLoopBackOff — ডেটাবেস সংযোগ:** ডেটাবেস Service/DNS/credential যাচাই করুন; অ্যাপকে retry/backoff দিতে বলুন; ডেটাবেস Pod-এর অবস্থা দেখুন।
4. **OOMKilled (137):** মেমরি limit বাড়ান, অ্যাপের মেমরি ব্যবহার পরীক্ষা করুন, প্রয়োজনে request/limit সমন্বয় করুন।
5. **Endpoints খালি:** Service-এর selector ও Pod-এর label মেলান; `kubectl get pods --show-labels` দিয়ে যাচাই করুন।
6. **DNS ব্যর্থতা:** CoreDNS Pod ও Service যাচাই করুন, Pod-এর `/etc/resolv.conf` দেখুন, NetworkPolicy DNS (53) বন্ধ করছে কি না দেখুন।
7. **Ingress 404:** host/path rule, Service-এর নাম ও পোর্ট মেলান; `kubectl describe ingress` ও controller লগ দেখুন।
8. **Ingress 502:** Ingress-এর পোর্ট Service-এর পোর্টের সঙ্গে মেলান (8080), readiness probe ঠিক করুন।
9. **PVC Pending:** StorageClass আছে কি না, provisioner চলছে কি না, PV আছে কি না যাচাই করুন; `kubectl describe pvc` দেখুন।
10. **Node NotReady:** নোডে disk/memory চাপ, kubelet ও runtime-এর লগ দেখুন; প্রয়োজনে cordon/drain করে মেরামত করুন।
11. **Rollout আটকে গেছে:** নতুন Pod-এর readiness probe ও লগ দেখুন; সমস্যা না মিটলে `kubectl rollout undo` করুন।
12. **RBAC Forbidden:** ServiceAccount-এর জন্য সঠিক namespace-এ Role ও RoleBinding তৈরি করুন; `can-i` দিয়ে যাচাই করুন।
13. **ConfigMap বদলায়নি:** Env var Pod restart ছাড়া বদলায় না — Pod rollout করুন বা checksum annotation ব্যবহার করুন।
14. **HPA unknown:** metrics-server ইনস্টল করুন ও Pod-এ CPU request দিন; `kubectl top pod` দিয়ে যাচাই করুন।
15. **NetworkPolicy:** অ্যাপের label Policy-র selector-এর সঙ্গে মেলান বা Policy-তে অ্যাপের label যোগ করুন।

16. **Secret key ভুল:** Secret-এর আসল key নামের সঙ্গে Deployment-এর `secretKeyRef` মেলান।
17. **StatefulSet ডেটা হারানো:** emptyDir-এর বদলে volumeClaimTemplates দিয়ে PVC ব্যবহার করুন; ডেটা পুনরুদ্ধারের জন্য ব্যাকআপ যাচাই করুন।
18. **etcd disk পূর্ণ:** পুরনো revision compact/defrag করুন, disk বাড়ান, snapshot নিয়ে পুনরুদ্ধারের পরিকল্পনা করুন; নিয়মিত monitoring দিন।
19. **Helm rollback ব্যর্থ:** হাতে বদলানো resource-ই কারণ — GitOps/Helm-এর বাইরে বদলাবেন না; `helm upgrade` দিয়ে সঠিক মান ফিরিয়ে আনুন।
20. **GitOps OutOfSync:** হাতে scale করা বদল Git-এ commit করুন, তারপর sync করুন; ভবিষ্যতে সব বদল Git-এ করুন।

কেস স্টাডি ১–১০: শিক্ষা

1. **ভুল ইমেজ ট্যাগ:** `:latest` নয়, নির্দিষ্ট tag ব্যবহার করুন; rollback পরিকল্পনা ও staging পরীক্ষা রাখুন; রাতে একা deploy নয়।
2. **Resource limit:** সব workload-এ request/limit দিন; LimitRange ও ResourceQuota দিয়ে ক্লাস্টার স্তরে সীমা বেঁধে দিন।
3. **Secret ফাঁস:** Secret কখনো Git-এ নয়; ফাঁস হলে সব credential বদলান, লগ পরীক্ষা করুন, pre-commit স্ক্যান ও Sealed/External Secrets ব্যবহার করুন।
4. **etcd ব্যাকআপ:** আপগ্রেডের আগে snapshot নিন ও পুনরুদ্ধার পরীক্ষা করুন; ব্যাকআপ ছাড়া আপগ্রেড করবেন না।
5. **Ingress পোর্ট:** Service পোর্ট ও Ingress পোর্ট মেলান; deploy-এর আগে staging-এ বাইরে থেকে পরীক্ষা করুন; স্বয়ংক্রিয় smoke test রাখুন।
6. **HPA ছাড়া সেল:** HPA, load test, PDB ও পর্যাপ্ত resource আগে থেকে প্রস্তুত রাখুন; হাতে স্কেল করা জরুরি অবস্থার সমাধান নয়।
7. **Cluster-admin:** Least privilege RBAC দিন; prod namespace মুছে ফেলা থেকে রক্ষায় আলাদা অনুমতি ও অনুমোদন প্রক্রিয়া রাখুন।
8. **PVC পূর্ণ:** Disk ব্যবহারের alert দিন; StorageClass-এ volume expansion চালু রাখুন; ক্ষমতা পরিকল্পনা করুন।
9. **GitOps-এর বাইরে বদল:** Git-ই একমাত্র উৎস; জরুরি বদলও Git-এ commit করে sync করুন।
10. **Node reboot:** সবসময় cordon → drain → রক্ষণাবেক্ষণ → uncordon ক্রম মানুন; গুরুত্বপূর্ণ Pod-এর একাধিক replica ও PDB রাখুন।

ফাইনাল পরীক্ষার নম্বর দেওয়ার নির্দেশিকা

অংশ A: MCQ-তে সঠিক উত্তরে ১ নম্বর; সংক্ষিপ্ত প্রশ্নে মূল পয়েন্ট (কারণ, প্রভাব, প্রতিকার) থাকলে পূর্ণ নম্বর। অংশ B/C: YAML সঠিকভাবে apply হয়েছে ও `kubectl get` আউটপুট মিলেছে কি না দেখুন — শুধু ফাইল লিখলে অর্ধেক নম্বর। অংশ D: কারণ নির্ণয়ের ব্যাখ্যা সঠিক হলে পূর্ণ নম্বর; শুধু rollback করলে অর্ধেক। ভুল image tag-এর ক্ষেত্রে `describe` ও `logs`-এর উল্লেখ থাকতে হবে।

Capstone গ্রেডিং রুব্রিক — সংক্ষেপ

মানদণ্ড	ওজন	উত্তীর্ণের চিহ্ন

আর্কিটেকচার ও নকশা	২০%	namespace, Deployment, Service, Ingress-এর যুক্তিযুক্ত বিভাজন
কনফিগারেশন ও নিরাপত্তা	২০%	ConfigMap/Secret, RBAC, Pod Security, NetworkPolicy সঠিক
নির্ভরযোগ্যতা	২০%	probe, resource, HPA, PDB, rollback পরীক্ষা করা
Storage ও ডেটা	১৫%	PVC, ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা
অটোমেশন (Helm/Kustomize/GitOps)	১৫%	পুনরুৎপাদনযোগ্য deploy, Git-ভিত্তিক পরিবর্তন
ডকুমেন্টেশন ও উপস্থাপনা	১০%	সিদ্ধান্তের কারণ, ট্রাবলশুটিং লগ, সীমাবদ্ধতা লেখা

Capstone পাস করতে মোট ৬০% এবং নিরাপত্তা অংশে অন্তত ৫০% পেতে হবে।